

A1:RESEARCH QUESTION

Can we use the ARIMA time series analysis machine learning model on teleco historical data so as to understand the current trends based on historical patterns?

A2: OBJECTIVES OR GOAL

The objective of this data analysis is to develop an ARIMA time series analysis model using the Python programming language to analyze the patterns in the Teleco telecommunication revenue data for the first two years of operations, and provide the executives of the company with the patterns that will be revealed in the time series analysis and the can use the patterns to understand the impact that the Churn in the current season.

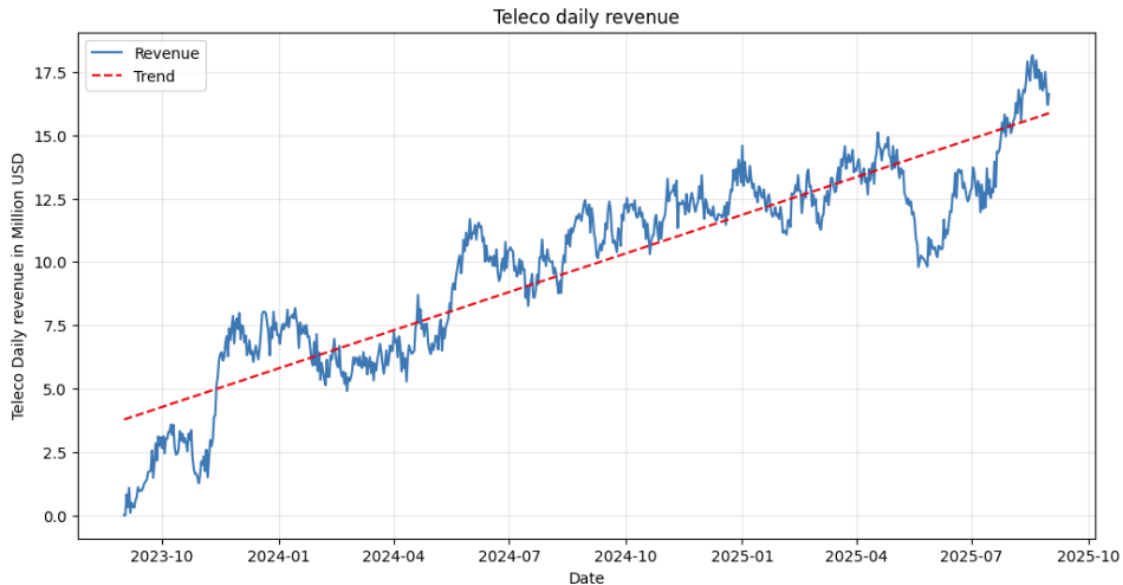
B2: SUMMARY OF ASSUMPTIONS

According to Intellectus Consulting (2025), a time series model has the following assumptions:

- **Stationarity:** The time series model assumes that the statistical properties of the series do not change over time, meaning that the data should be stationary in nature with no trend or seasonality, and the time series follows a normal distribution with a constant mean and variance over time.
- **No outliers:** The time series model assumes that there are no outliers in the data series, as outliers may affect the time series and can be misleading and distort the results.
- **Uncorrelated error terms(White Noise):** The time series model assumes that the error component (Random shocks) is independently and randomly distributed with a mean of zero and a constant variance.
- **Correlation between observations at different time lags.** A time series model assumes that past patterns, trends, and relationships will persist in the future.

C1: LINE GRAPH VISUALIZATION

The line graph shows the trend of the revenues for the teleco company for the last two years.



C2: TIME STEP FORMATTING

Time step formatting refers to how time intervals are represented and structured in a time series dataset. The frequency of the teleco series data is daily.

```
#Teleco_df time step formatting
Teleco_df=Teleco_df.asfreq('D')
print(Teleco_df.index)
print(f"Start date: {Teleco_df.index.min()}")
print(f"End date: {Teleco_df.index.max()}")
print(f"Total days: {len(Teleco_df)}")
print(f"Frequency: {Teleco_df.index.freqstr}")

DatetimeIndex(['2023-09-01', '2023-09-02', '2023-09-03', '2023-09-04',
               '2023-09-05', '2023-09-06', '2023-09-07', '2023-09-08',
               '2023-09-09', '2023-09-10',
               ...,
               '2025-08-22', '2025-08-23', '2025-08-24', '2025-08-25',
               '2025-08-26', '2025-08-27', '2025-08-28', '2025-08-29',
               '2025-08-30', '2025-08-31'],
              dtype='datetime64[ns]', name='Date', length=731, freq='D')
Start date: 2023-09-01 00:00:00
End date: 2025-08-31 00:00:00
Total days: 731
Frequency: D
```

Gaps in measurement occur when the expected time steps are missing from the dataset. In the case of Teleco data, the gap measurement is zero, and hence, there is continuity of the time series data.

```
#Detecting gaps
Teleco_gaps=Teleco_df[Teleco_df['Revenue'].isna()]
print(len(Teleco_gaps))
```

0

The length of the sequence refers to the total number of time steps or observations in the time series. The Teleco dataset has 731 observations

```
#Teleco Sequence Length
Teleco_Sequence_Lenth = len(Teleco_df)
print(Teleco_Sequence_Lenth)
```

731

C3:STATIONARITY

Stationarity means that the time series behaves consistently over time. Stationarity can be assessed by the mean, the variance, and the correlation. To assess the stationarity of Teleco data, I performed an Augmented Dickey-Fuller (ADF) test, which shows that the trend of the Teleco data is non-stationary with a p-value of 0.32 and an ADF Statistics value of -1.92. This confirms the spiking that was observed in the line graph and also reflected by the rolling mean and standard deviation.

```
#Stationarity test using the ADF test. Using the AIC autolag on the original data
Stationarity_Results = adfuller(Teleco_df['Revenue'].dropna())
```

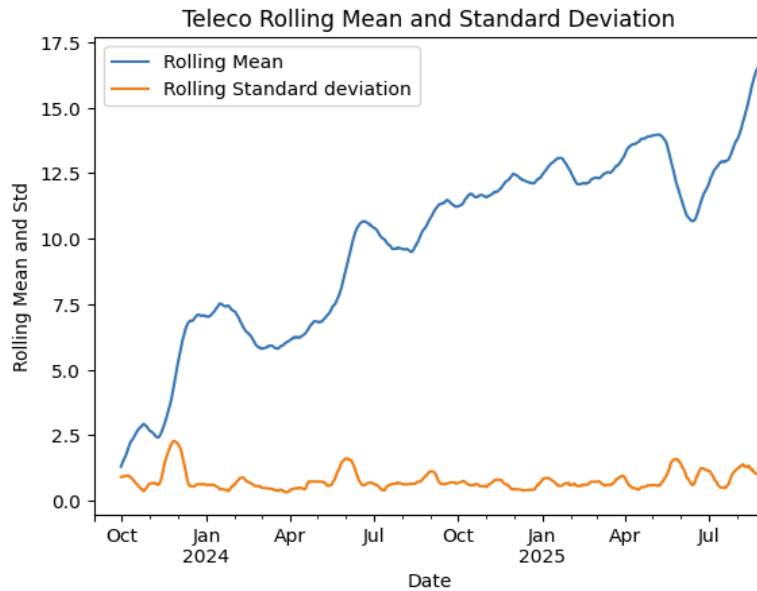
```
labels = [
    "ADF Statistic",
    "p-value",
    "Used lags",
    "Number of observations",
    "Critical Values",
    "IC Best"
]

print("# Stationarity Test (ADF) Original data")
for label, value in zip(labels, Stationarity_Results):
    if label == "Critical Values":
        print(f"{label}:")
        for k, v in value.items():
            print(f"    {k}: {v:.4f}")
    else:
        print(f"{label}: {value}")
```

```
# Stationarity Test (ADF) Original data
ADF Statistic: -1.9246121573101838
p-value: 0.3205728150793964
Used lags: 1
Number of observations: 729
Critical Values:
  1%: -3.4394
  5%: -2.8655
 10%: -2.5689
IC Best: 965.0609576707513
```

```
#Confirming the non stationarity using the rolling means
rolling_mean = Teleco_df['Revenue'].rolling(window=30).mean()
rolling_std = Teleco_df['Revenue'].rolling(window=30).std()

rolling_mean.plot(label='Rolling Mean')
rolling_std.plot(label='Rolling Standard deviation')
plt.ylabel("Rolling Mean and Std")
plt.legend()
plt.title('Teleco Rolling Mean and Standard Deviation')
plt.show()
```



C4:STEPS TO PREPARE THE DATA

In preparing the data for analysis, the following steps were taken

- I loaded the data from a CSV into a pandas data frame and conducted initial data exploration.

```
# Loading the data
# File path variables

Local_path = 'C:/Users/beatr/OneDrive/Desktop/WGU/D213/Task1/Data/'

InputFilePath = Local_path + 'teleco_time_series.csv'
CleanedFilePath = Local_path + 'Teleco_cleaned_time_series.csv'

TrainedDataFilePath = Local_path + 'Teleco_Train_data.csv'
TestDataFilePath = Local_path + 'Teleco_Test_data.csv'
Teleco_df = pd.read_csv(InputFilePath)

# Initial data exploration (Understanding the data)
pd.set_option('display.max_columns', None)
Teleco_df
```

```
In [ ]:
      Day  Revenue
0      1  0.000000
1      2  0.000793
2      3  0.825542
3      4  0.320332
4      5  1.082554
...    ...      ...
726    727 16.931559
727    728 17.490666
728    729 16.803638
729    730 16.194813
730    731 16.620798

731 rows x 2 columns
```

```
In [ ]: Teleco_df.shape
```

```
In [ ]: (731, 2)
```

```
In [ ]: Teleco_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 731 entries, 0 to 730
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0    Day    731 non-null    int64  
 1  Revenue  731 non-null    float64
dtypes: float64(1), int64(1)
memory usage: 11.6 KB
```

- Converted the day column into an appropriate datetime column to allow for time series forecasting. Created an index on the datetime column and sorted the data.

```
#Converting day into datetime, creating index and sorting the data

Teleco_df['Date']=pd.to_datetime(Teleco_df['Day'],unit='D', origin='2023/07/31')
Teleco_df=Teleco_df[['Date','Revenue']]
Teleco_df=Teleco_df.set_index("Date")
Teleco_df=Teleco_df.sort_index()
Teleco_df = Teleco_df.asfreq("D")
Teleco_df = Teleco_df.asfreq(pd.infer_freq(Teleco_df.index))

print(Teleco_df.info())
Teleco_df
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 731 entries, 2023-08-01 to 2025-07-31
Freq: D
Data columns (total 1 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   Revenue 731 non-null    float64
dtypes: float64(1)
memory usage: 11.4 KB
None
```

```
]:
```

Revenue	
Date	
2023-08-01	0.000000
2023-08-02	0.000793
2023-08-03	0.825542
2023-08-04	0.320332
2023-08-05	1.082554
...	...
2025-07-27	16.931599
2025-07-28	17.490886
2025-07-29	16.803838
2025-07-30	16.194813
2025-07-31	16.620798

731 rows × 1 columns

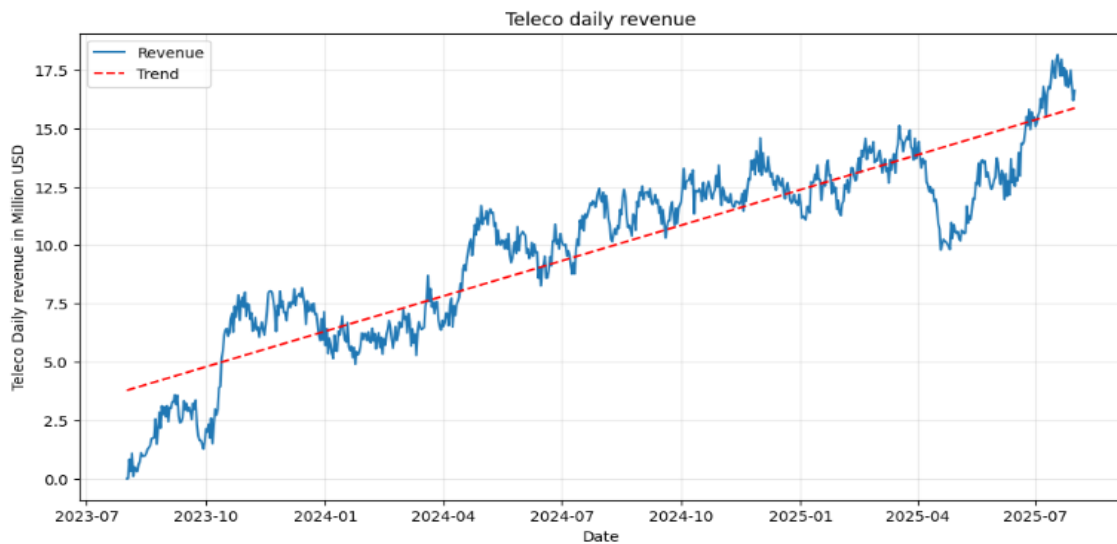
- Created a line graph to check for trend, seasonality, and outliers

```
#LINE GRAPH VISUALIZATION
#Creating the initial time series graph to check for trend, seasonality,outlier
plt.figure(figsize=(12,6))
plt.plot(Teleco_df.index, Teleco_df, label="Revenue")

# Fitting the best line (polynomial) to the data
x = np.arange(len(Teleco_df['Revenue'].values))
y = Teleco_df['Revenue'].values

z = np.polyfit(x, y, 1)
p = np.poly1d(z)
plt.plot(Teleco_df.index, p(x), color='red',linestyle='--',label='Trend')

plt.title("Teleco daily revenue")
plt.ylabel("Teleco Daily revenue in Million USD")
plt.xlabel("Date")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```



- Checked if there are any gaps in the data and nulls

```
#Detecting gaps
Teleco_gaps=Teleco_df[Teleco_df['Revenue'].isna()]
print(len(Teleco_gaps))
```

0

```
#Teleco Sequence Length
Teleco_Sequence_Lenth = len(Teleco_df)
print(Teleco_Sequence_Lenth)
```

731

- Checked for stationarity in the original data

```
#Stationarity test using the ADF test. Using the AIC autolag on the original data
Stationarity_Results = adfuller(Teleco_df['Revenue'].dropna())

labels = [
    "ADF Statistic",
    "p-value",
    "Used lags",
    "Number of observations",
    "Critical Values",
    "IC Best"
]

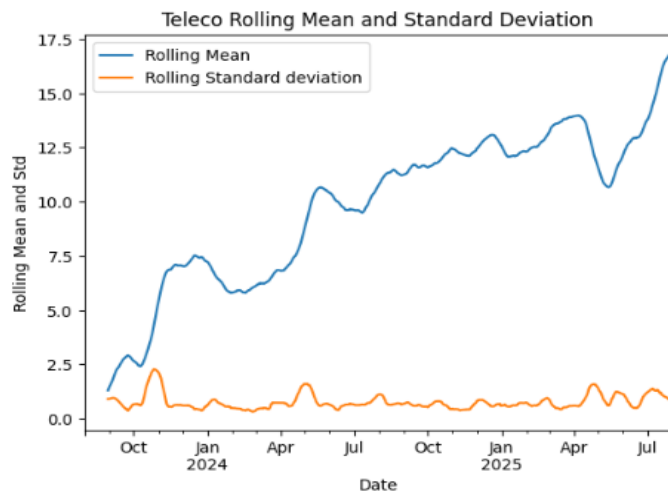
print("# Stationarity Test (ADF) Original data")
for label, value in zip(labels, Stationarity_Results):
    if label == "Critical values":
        print(f"{label}:")
        for k, v in value.items():
            print(f"    {k}: {v:.4f}")
    else:
        print(f"{label}: {value}")

# Stationarity Test (ADF) Original data
ADF Statistic: -1.9246121573101838
p-value: 0.3205728150793964
Used lags: 1
Number of observations: 729
Critical Values:
 1%: -3.4394
 5%: -2.8655
10%: -2.5689
IC Best: 965.0609576707513
```

- Confirmed non-stationarity

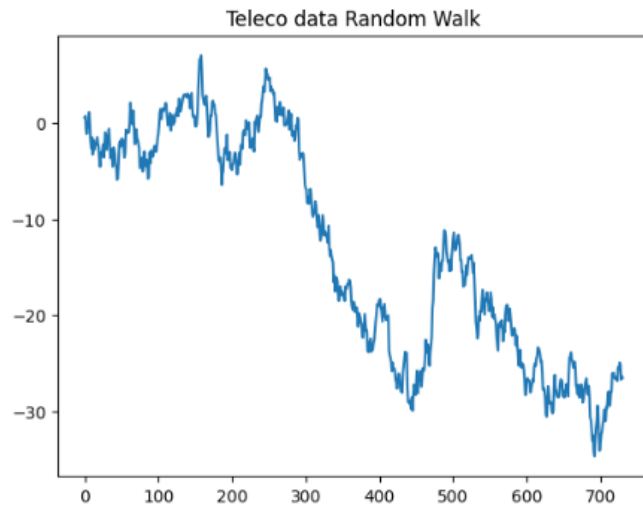
```
#Confirming the non stationarity using the rolling means
rolling_mean = Teleco_df['Revenue'].rolling(window=30).mean()
rolling_std = Teleco_df['Revenue'].rolling(window=30).std()

rolling_mean.plot(label='Rolling Mean')
rolling_std.plot(label='Rolling Standard deviation')
plt.ylabel("Rolling Mean and Std")
plt.legend()
plt.title('Teleco Rolling Mean and Standard Deviation')
plt.show()
```



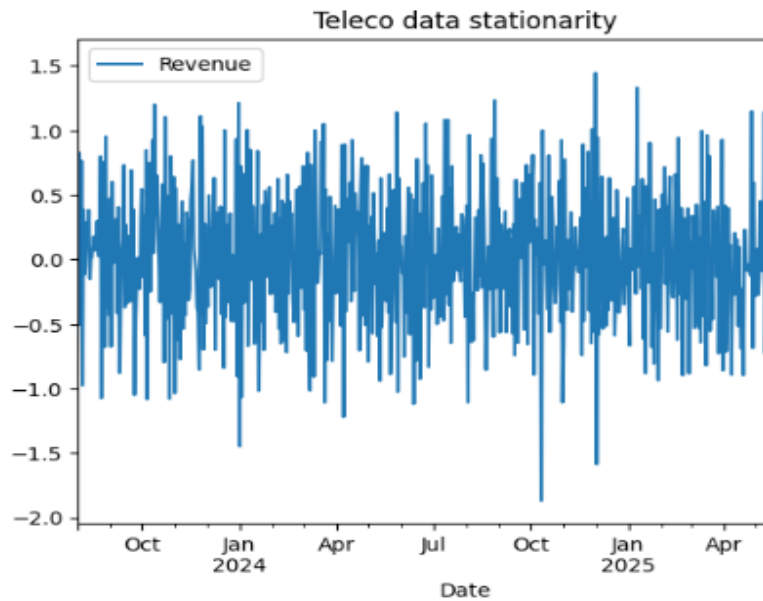
- Checked for the randomness of the data

```
#random walk  
#white noise is pure randomness. Random walk is a path where each step is random but depends on the previous one.  
random_walk = np.cumsum(np.random.normal(0, 1, len(Teleco_df)))  
plt.plot(random_walk)  
plt.title('Teleco data Random Walk')  
plt.show()
```



- Removed the trends and stabilized the mean using diff

```
#Data cleaning - Removing the trend/seasonality in the data  
#(removing the trend from the teleco dataset to make it stationar  
Teleco_df_diff=Teleco_df.diff().dropna()  
Teleco_df_diff.plot()  
plt.title('Teleco data stationarity')  
plt.show()
```



```
#Rerunning the ADF Fuller on the differenced data
```

- Checked for stationarity in the differenced data

```
#Rerunning the ADF Fuller on the differenced data
#Stationarity test using the ADF test
Stationarity_Results = adfuller(Teleco_df_diff['Revenue'].dropna())

labels = [
    "ADF Statistic",
    "p-value",
    "Used lags",
    "Number of observations",
    "Critical Values",
    "IC Best"
]

print("# Stationarity Test (ADF)-Differenced data")
for label, value in zip(labels, Stationarity_Results):
    if label == "Critical Values":
        print(f"{label}:")
        for k, v in value.items():
            print(f"    {k}: {v:.4f}")
    else:
        print(f"{label}: {value}")

# Stationarity Test (ADF)-Differenced data
ADF Statistic: -44.87452719387598
p-value: 0.0
Used lags: 0
Number of observations: 729
Critical Values:
    1%: -3.4394
    5%: -2.8655
    10%: -2.5689
IC Best: 965.5032159185916
```

```
#Splitting the data into training and testing data
```

- Split the data into train and test data sets with a 20%split

```
#Splitting the data into training and testing data
```

```
train_data,test_data=train_test_split(Teleco_df_diff,test_size=0.2,random_state=42, shuffle=False)  
train_data.head(5)
```

Date	Revenue
2023-08-02	0.000793
2023-08-03	0.824749
2023-08-04	-0.505210
2023-08-05	0.762222
2023-08-06	-0.974900

```
test_data.head()
```

Date	Revenue
2025-03-08	-0.531923
2025-03-09	0.157387
2025-03-10	-0.644689
2025-03-11	0.995057
2025-03-12	-0.438775

```
#Outputing the cleandata to csv  
Teleco_df_diff.to_csv(CleanedFilePath)  
train_data.to_csv(TrainedDataFilePath)  
test_data.to_csv(TestDataFilePath)
```

- Created an output of the cleaned data, and the test and train datasets

```
#Splitting the data into training and testing data
```

```
train_data,test_data=train_test_split(Teleco_df_diff,test_size=0.2,random_state=42, shuffle=False)  
train_data.head(5)
```

Date	Revenue
2023-08-02	0.000793
2023-08-03	0.824749
2023-08-04	-0.505210
2023-08-05	0.762222
2023-08-06	-0.974900

```
test_data.head()
```

Date	Revenue
2025-03-08	-0.531923
2025-03-09	0.157387
2025-03-10	-0.644689
2025-03-11	0.995057
2025-03-12	-0.438775

```
#Outputing the cleandata to csv  
Teleco_df_diff.to_csv(CleanedFilePath)  
train_data.to_csv(TrainedDataFilePath)  
test_data.to_csv(TestDataFilePath)
```

C5:PREPARED DATA SET

- Cleaned data



Teleco_cleaned_time_series.csv

- Train data



Teleco_Train_data.csv

- Test data



Teleco_Test_data.csv

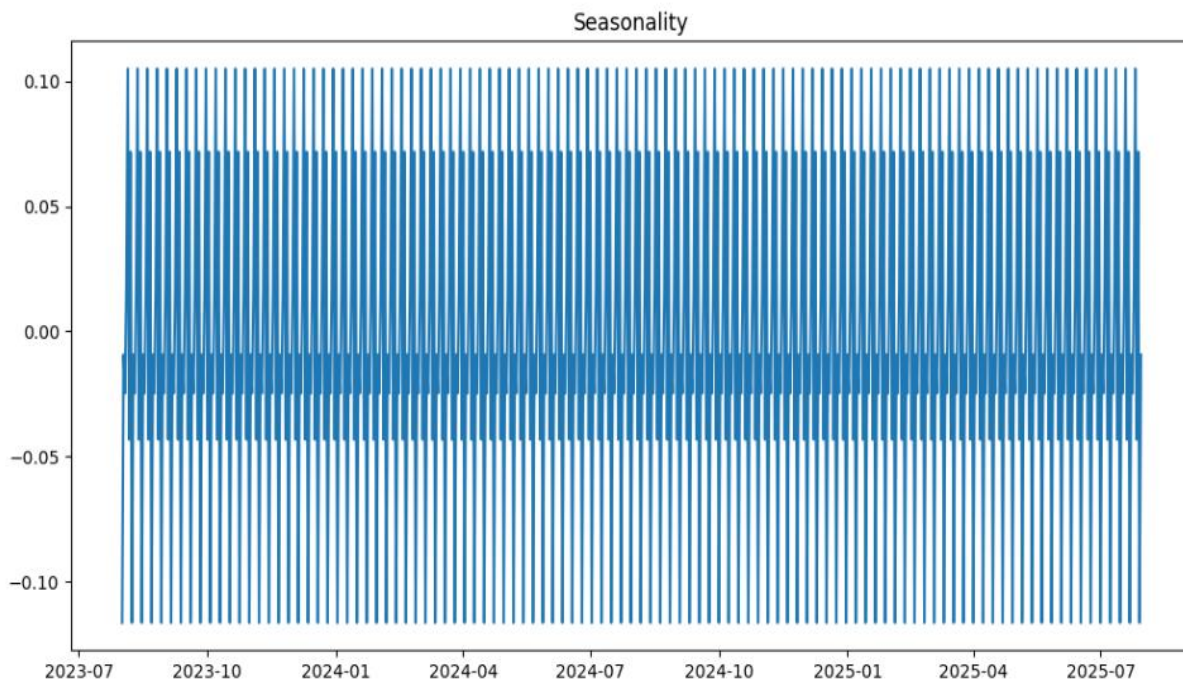
D1: REPORT FINDINGS AND VISUALIZATIONS

- the presence or lack of a seasonal component

Seasonality references patterns that repeat at regular intervals over time. This can be detected using the autocorrelation function. The seasonality captures repeating patterns at regular intervals. Teleco data shows cycles and seasonality in the Revenue generation.

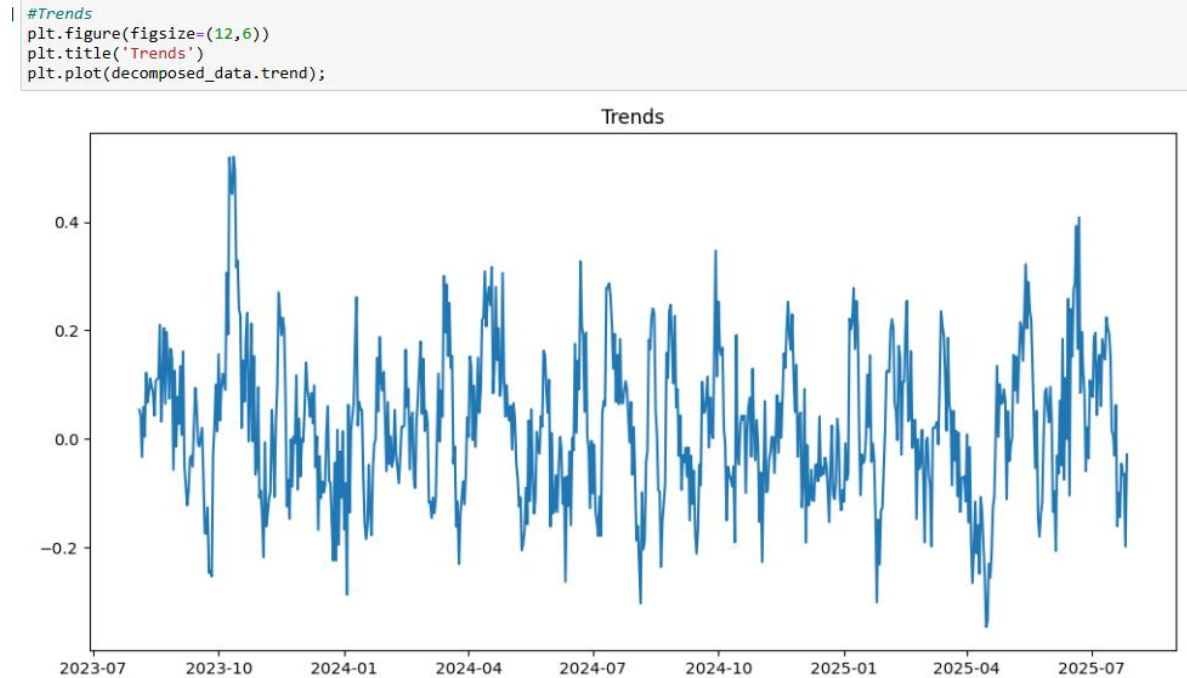
```
#Seasonality
#the presence or lack of a seasonal component
decomposed_data= seasonal_decompose(Teleco_df_diff)
plt.figure(figsize=(12,6))
plt.title('Seasonality')
plt.plot(decomposed_data.seasonal)
```

[<matplotlib.lines.Line2D at 0x20972a5fc50>]



- trends

The Trend represents the long-term progression of the data and shows if it is increasing, decreasing, or staying flat over time. In the case of Teleco data, the trend shows that the revenue is slowly rising.



- the autocorrelation function

The autocorrelation function measures how the current values relate to past values at different lags. A partial autocorrelation is a summary of the relationship between an observation in a time series with observations at prior time steps, with the relationship of intervening observations removed.

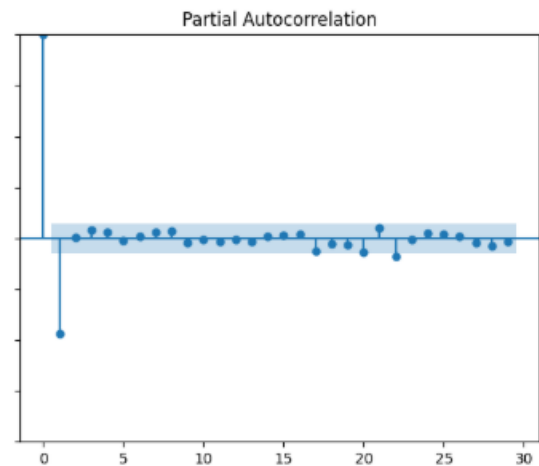
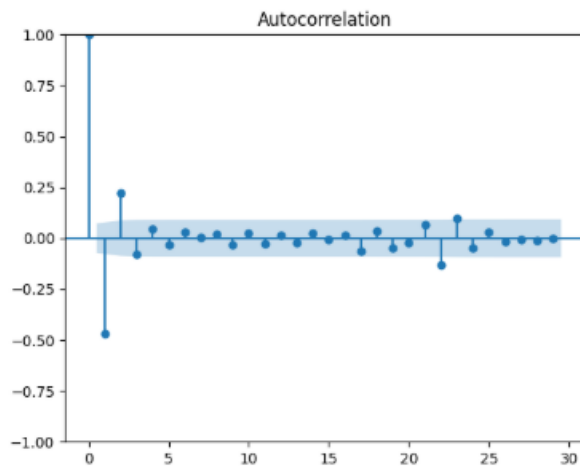
```
#autocorrelation function
print('Autocorrelation function')
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=[14,5], sharey=True)
plot_acf(Teleco_df_diff,ax=ax1)
plot_pacf(Teleco_df_diff,ax=ax2)
plt.show()

Acf_Value=acf(Teleco_df_diff)
Pacf_Value=pacf(Teleco_df_diff)

Teleco_lags=list(range(len(Acf_Value)))
Acf_Pacf_df= pd.DataFrame({'Lags':Teleco_lags, 'ACF':Acf_Value, 'PACF':Pacf_Value})

print(Acf_Pacf_df)
```

Autocorrelation function



	Lags	ACF	PACF
0	0	1.000000	1.000000
1	1	-0.469506	-0.470150
2	2	0.223874	0.004427
3	3	-0.076620	0.038797
4	4	0.047843	0.032467
5	5	-0.034508	-0.009443
6	6	0.029196	0.007075
7	7	0.004326	0.029870
8	8	0.017328	0.036432
9	9	-0.030830	-0.019598
10	10	0.023792	-0.006319
11	11	-0.027709	-0.018397
12	12	0.013641	-0.007650
13	13	-0.020448	-0.017175
14	14	0.022476	0.007463
15	15	-0.007156	0.013359
16	16	0.016461	0.021423
17	17	-0.064114	-0.065633
18	18	0.035488	-0.029407
19	19	-0.048303	-0.035548
20	20	-0.021149	-0.071277
21	21	0.068434	0.054827
22	22	-0.130043	-0.092892
23	23	0.095537	-0.008714
24	24	-0.046111	0.025421
25	25	0.028504	0.021075
26	26	-0.015962	0.007968
27	27	-0.008878	-0.025269
28	28	-0.012601	-0.036867

Based on the calculated ACF using the differenced data (after removing the seasonality and trends), at lag 1, the ACF value is -0.469, showing a strong negative autocorrelation, and at lag 2, the ACF value is +0.223, which is a positive weak autocorrelation. Past lag 2, the ACF values are low and within the confidence interval, as shown on the Autocorrelation function plot. This suggests that the moving average component (q) MA is 1. The moving average component helps in correcting for short-term noise in the data. It captures the relationship between a time series and the residual errors from past forecasts.

On the other hand, the PACF calculated on the differenced data, at lag 1, is -0.470, showing a strong negative partial autocorrelation. At lag 2 and beyond, the partial correlation is negligible and within the confidence interval, as shown in the plot. This indicates that the autoregressive

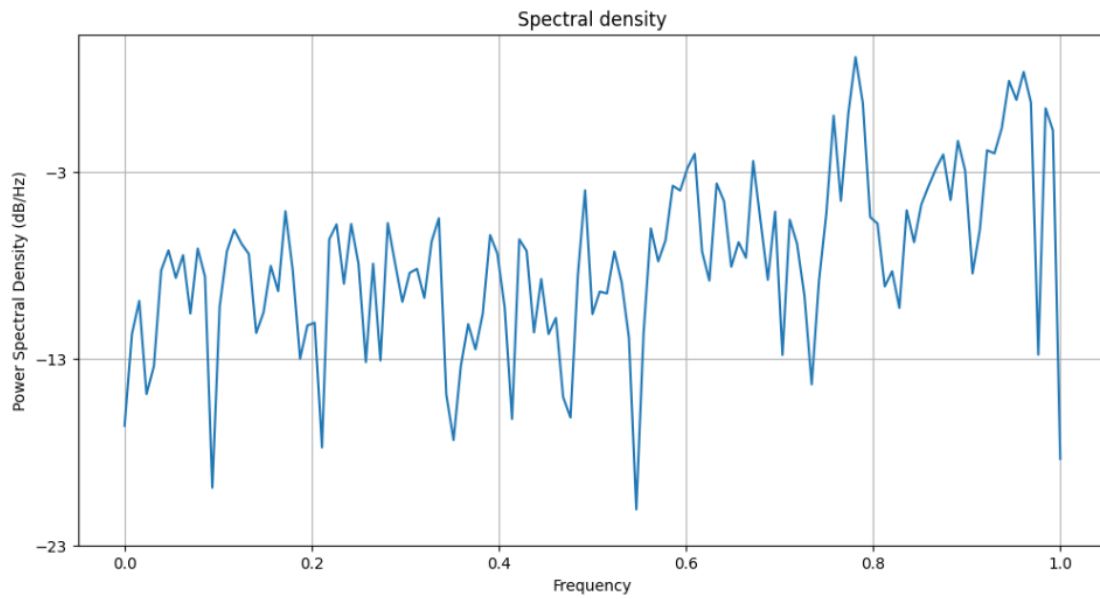
component (p) AR is 1. The AR is chosen based on how many past values significantly affect the current value. It captures the idea that past values influence the current values. The AR assumes that the time series can be explained by a linear combination of its own previous values.

- the spectral density

The spectral density helps identify cyclical patterns or periodic behavior by revealing the hidden cycles. Spectral density breaks time series into its frequency components, showing how much power or variance is associated with each frequency and which frequencies dominate the signals. It also helps distinguish meaningful oscillations from random fluctuations.

Based on the diagram below from the Teleco data, we are able to observe the hidden patterns in the data which the executives can use to understand the current business performance. More so, there is a variance in each frequency component, with the dominant behavior being oscillatory and structured.

```
# the spectral density  
plt.figure(figsize=(12,6))  
plt.title('Spectral density')  
plt.psd(x=Teleco_df_diff.Revenue)  
plt.show()
```



Teleco Spectral dataframe

	Frequency	Power
0	0.000000	0.000000
1	0.001370	0.107302
2	0.002740	0.179280
3	0.004110	0.177296
4	0.005479	0.012843
5	0.006849	0.408582
6	0.008219	0.235838
7	0.009589	0.201203
8	0.010959	0.090099
9	0.012329	0.272280
10	0.013699	0.057737
11	0.015068	0.123510
12	0.016438	0.058434
13	0.017808	0.572680
14	0.019178	0.775726
15	0.020548	0.496580
16	0.021918	0.385218
17	0.023288	0.429538
18	0.024658	0.501061
19	0.026027	0.179268
20	0.027397	0.026811
21	0.028767	0.155080
22	0.030137	0.227005
23	0.031507	0.398160
24	0.032877	0.444933
25	0.034247	0.292125
26	0.035616	0.021805
27	0.036986	0.135135
28	0.038356	0.149711
29	0.039726	0.609167
30	0.041096	0.369967
31	0.042466	0.168094
32	0.043836	0.153662
33	0.045205	0.044419
34	0.046575	0.041769
35	0.047945	0.164504
36	0.049315	0.423515
37	0.050685	0.058710
38	0.052055	0.098564
39	0.053425	0.136206
40	0.054795	0.378591
41	0.056164	0.046083
42	0.057534	0.016376
43	0.058904	0.576588
44	0.060274	0.299058
45	0.061644	0.340084
46	0.063014	0.022623
47	0.064384	0.048531
48	0.065753	0.393483
49	0.067123	0.261845
50	0.068493	0.120286
51	0.069863	0.029293
52	0.071233	0.102531
53	0.072603	0.049942
54	0.073973	0.146689
55	0.075342	0.451301
56	0.076712	0.027756
57	0.078082	0.044568
58	0.079452	0.526101
59	0.080822	0.348019

- the decomposed time series

A decomposed time series comprises the trend, seasonality, and Residual or noise.

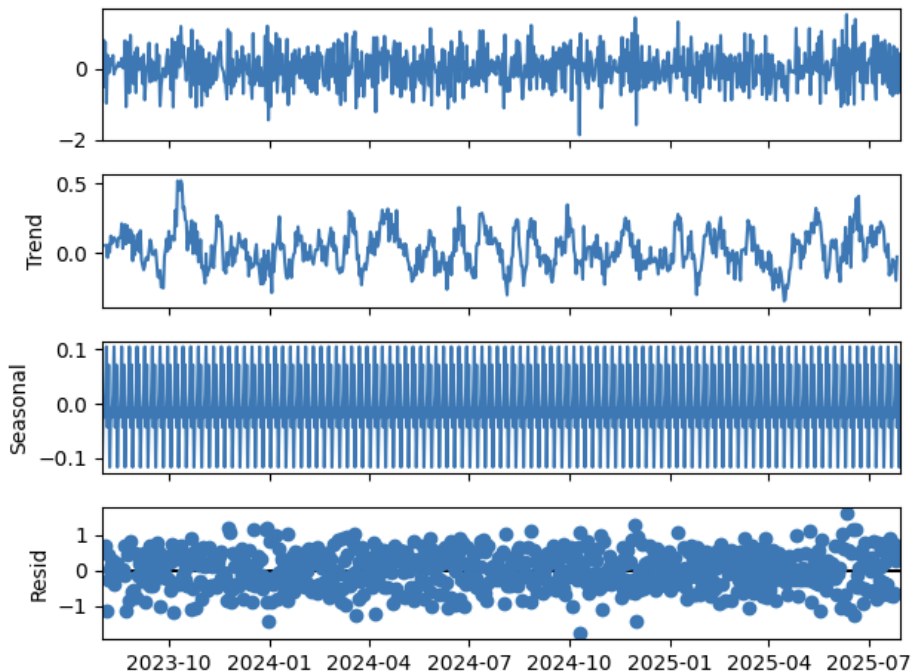
The Trend represents the long-term progression of the data and shows if it is increasing, decreasing, or staying flat over time. In the case of Teleco data, the trend shows that the revenue is slowly rising.

The seasonality captures repeating patterns at regular intervals. Teleco data shows cycles and seasonality in the Revenue generation.

Residual or Noise shows the random fluctuations, anomalies, or unexplained variations in the data after removing the trend and seasonality. The Teleco data shows small spikes and irregular noises in the low power frequencies, and irregular dips

```
#the decomposed time series  
plt.figure(figsize=(18,10))  
decomposed_data.plot()  
plt.show()
```

<Figure size 1800x1000 with 0 Axes>

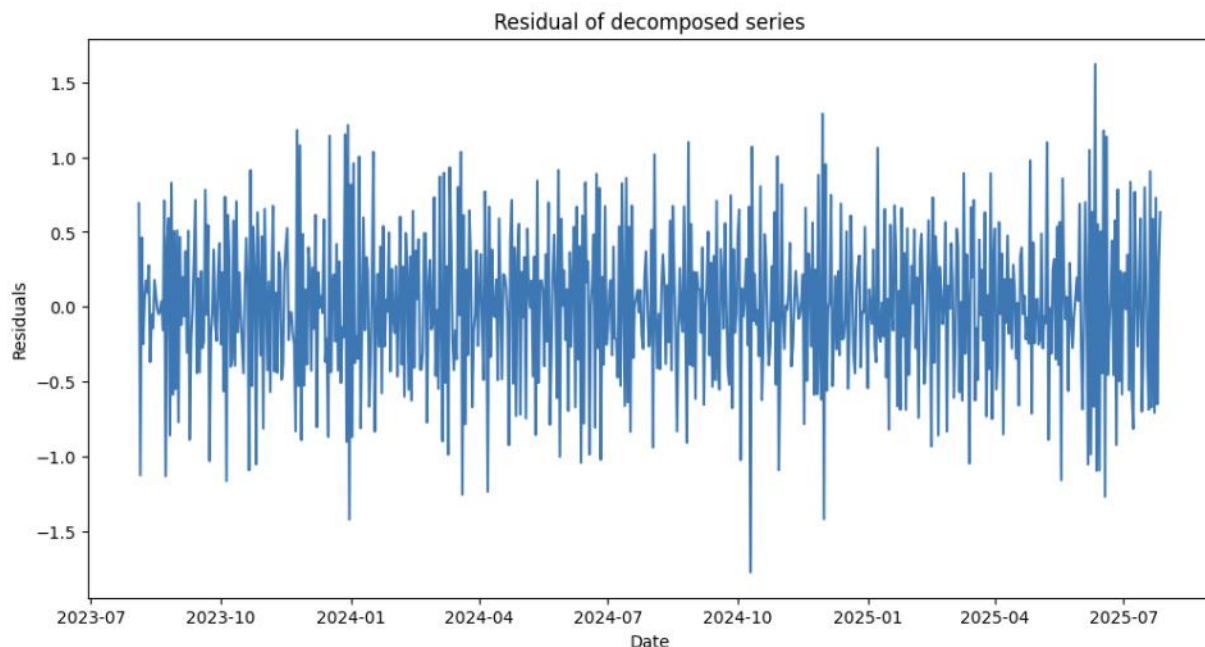


- confirmation of the lack of trends in the residuals of the decomposed series

The Trend represents the long-term progression of the data and shows if it is increasing, decreasing, or staying flat over time. In the case of Teleco data, the trend shows that the revenue is slowly rising. The decomposed series shows trends in the residual at the peaks and in the dips.

```
#confirmation of the lack of trends in the residuals of the decomposed series
plt.figure(figsize=(12,6))
plt.title('Residual of decomposed series')
plt.xlabel("Date")
plt.ylabel("Residuals")
plt.plot(decomposed_data.resid)
```

```
[<matplotlib.lines.Line2D at 0x209725575d0>]
```



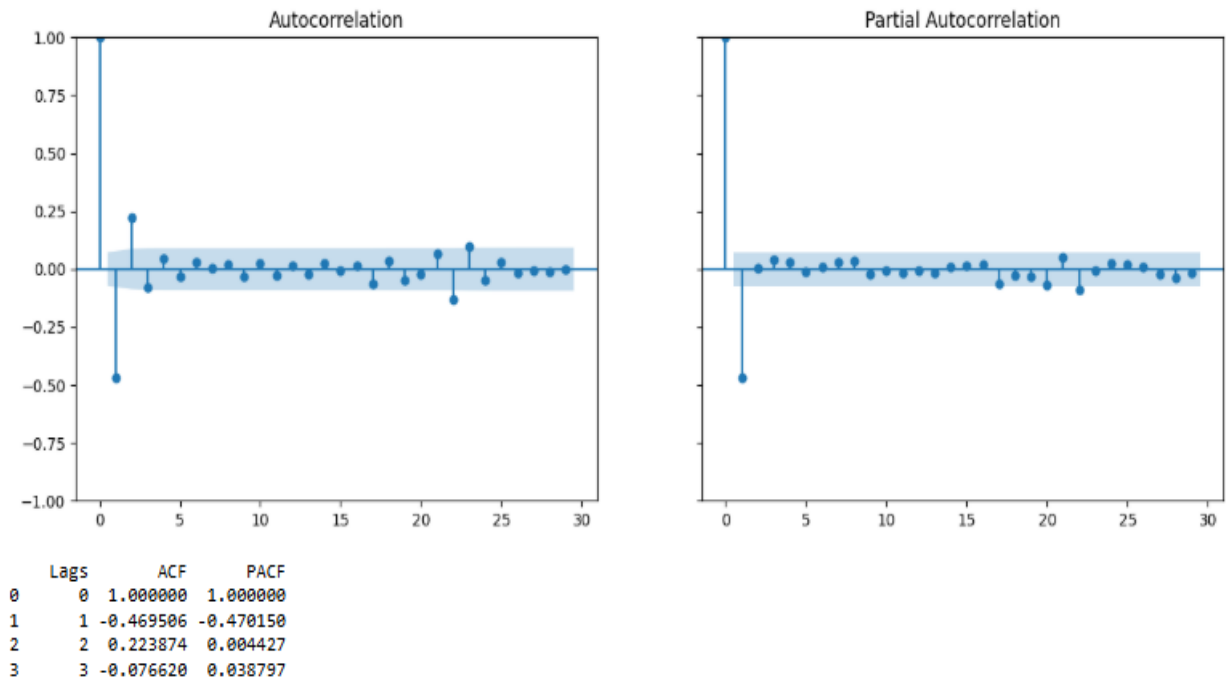
D2: ARIMA MODEL

An autoregressive integrated moving average (ARIMA) model is a statistical model that is used to analyze and forecast time series data by capturing the patterns in the data, such as the trend and the autocorrelation. It comprises (p,d,q), where p represents the autoregressive terms(AR), d represents the number of times the data is differenced(I), and q represents the number of moving averages(MA).

From the Teleco dataset provided, the PACF identified a lag of 1, meaning that an AR of 1 past value is used to predict future value.

The ACF identified a Lag of 1, hence the MA of 1 is used as the past forecast errors to improve predictions

Autocorrelation function



Running an ADF Fuller test on the Teleco differenced data shows a lag of 0, whereas the original data had a lag of 1. Meaning that one round of differencing was sufficient to make the time series stationary, hence $d=1$

```
#Stationarity test using the ADF test. Using the AIC autolag on the original data  
Stationarity_Results = adfuller(Teleco_df['Revenue'].dropna())
```

```
labels = [  
    "ADF Statistic",  
    "p-value",  
    "Used lags",  
    "Number of observations",  
    "Critical Values",  
    "IC Best"  
]  
  
print("# Stationarity Test (ADF)")  
for label, value in zip(labels, Stationarity_Results):  
    if label == "Critical Values":  
        print(f"{label}:")  
        for k, v in value.items():  
            print(f"    {k}: {v:.4f}")  
    else:  
        print(f"{label}: {value}")
```

```
# Stationarity Test (ADF)  
ADF Statistic: -1.9246121573101838  
p-value: 0.3205728150793964  
Used lags: 1  
Number of observations: 729  
Critical Values:  
    1%: -3.4394  
    5%: -2.8655  
    10%: -2.5689  
IC Best: 965.0609576707513
```



```
#Rerunning the ADF Fuller on the differenced data
#Stationarity test using the ADF test
Stationarity_Results = adfuller(Teleco_df_diff['Revenue'].dropna())

labels = [
    "ADF Statistic",
    "p-value",
    "Used lags",
    "Number of observations",
    "Critical Values",
    "IC Best"
]

print("# Stationarity Test (ADF)-Differenced data")
for label, value in zip(labels, Stationarity_Results):
    if label == "Critical Values":
        print(f"{label}:")
        for k, v in value.items():
            print(f"    {k}: {v:.4f}")
    else:
        print(f"{label}: {value}")

# Stationarity Test (ADF)-Differenced data
ADF Statistic: -44.87452719387598
p-value: 0.0
Used lags: 0
Number of observations: 729
Critical Values:
  1%: -3.4394
  5%: -2.8655
 10%: -2.5689
IC Best: 965.5032159185916
```

Initial manual assessment of the ARIMA model, using the AR =1, I = 1, and MA =1, performed poorly and had higher AIC(780.70) and BIC (793.81), indicating over differencing, and the MA(1) was insignificant.

```
#Creating the initial model
Initial_model = ARIMA(train_data, order=(1,1,1))
Initial_model_fit = Initial_model.fit()
print(Initial_model_fit.summary())
```

SARIMAX Results

```
=====
Dep. Variable:      Revenue    No. Observations:      584
Model:              ARIMA(1, 1, 1)    Log Likelihood      -387.352
Date:              Tue, 16 Sep 2025    AIC                 780.703
Time:              19:49:11           BIC                 793.808
Sample:            08-02-2023         HQIC                785.811
                  - 03-07-2025
Covariance Type:    opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	-0.4587	0.037	-12.556	0.000	-0.530	-0.387
ma.L1	-0.9997	0.098	-10.191	0.000	-1.192	-0.807
sigma2	0.2184	0.024	9.004	0.000	0.171	0.266

```
=====
Ljung-Box (L1) (Q):      0.01    Jarque-Bera (JB):      1.93
Prob(Q):                 0.91    Prob(JB):              0.38
Heteroskedasticity (H):  0.96    Skew:                  -0.08
Prob(H) (two-sided):     0.75    Kurtosis:              2.76
=====
```

Warnings:

```
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

In the selection of the ARIMA model to use, auto_arima from the pmdarima library was used to dynamically determine the best model.

The dynamic ARIMA model (1,0,0) produced a better AIC of 774.99 and BIC of 783.73, confirming that the model is more parsimonious and explains the data well using the fewest parameters and avoids over-fitting of the data.

The ARIMA model using (1,0,0)

```
#Determining the best ARIMA model
Teleco_best_model = auto_arima(train_data, start_p=0, start_q=0, max_p=3, max_q=3, d=None,
                              seasonal=True, stepwise=True, information_criterion='bic', trace=True)
print(Teleco_best_model.summary())
```

```
Performing stepwise search to minimize bic
ARIMA(0,0,0)(0,0,0)[0] intercept : BIC=919.529, Time=0.06 sec
ARIMA(1,0,0)(0,0,0)[0] intercept : BIC=787.002, Time=0.05 sec
ARIMA(0,0,1)(0,0,0)[0] intercept : BIC=812.574, Time=0.08 sec
ARIMA(0,0,0)(0,0,0)[0] intercept : BIC=914.318, Time=0.04 sec
ARIMA(2,0,0)(0,0,0)[0] intercept : BIC=793.366, Time=0.07 sec
ARIMA(1,0,1)(0,0,0)[0] intercept : BIC=793.367, Time=0.08 sec
ARIMA(2,0,1)(0,0,0)[0] intercept : BIC=799.543, Time=0.49 sec
ARIMA(1,0,0)(0,0,0)[0] intercept : BIC=783.730, Time=0.03 sec
ARIMA(2,0,0)(0,0,0)[0] intercept : BIC=790.098, Time=0.03 sec
ARIMA(1,0,1)(0,0,0)[0] intercept : BIC=790.099, Time=0.05 sec
ARIMA(0,0,1)(0,0,0)[0] intercept : BIC=809.839, Time=0.06 sec
ARIMA(2,0,1)(0,0,0)[0] intercept : BIC=795.977, Time=0.15 sec
```

```
Best model: ARIMA(1,0,0)(0,0,0)[0]
Total fit time: 1.179 seconds
```

SARIMAX Results

```
=====
Dep. Variable: y No. Observations: 584
Model: SARIMAX(1, 0, 0) Log Likelihood: -385.495
Date: Tue, 16 Sep 2025 AIC: 774.990
Time: 18:20:19 BIC: 783.730
Sample: 08-02-2023 HQIC: 778.397
- 03-07-2025
Covariance Type: opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	-0.4568	0.036	-12.605	0.000	-0.528	-0.386
sigma2	0.2191	0.014	15.965	0.000	0.192	0.246

```
=====
Ljung-Box (L1) (Q): 0.02 Jarque-Bera (JB): 1.85
Prob(Q): 0.90 Prob(JB): 0.40
Heteroskedasticity (H): 0.96 Skew: -0.08
Prob(H) (two-sided): 0.79 Kurtosis: 2.77
=====
```

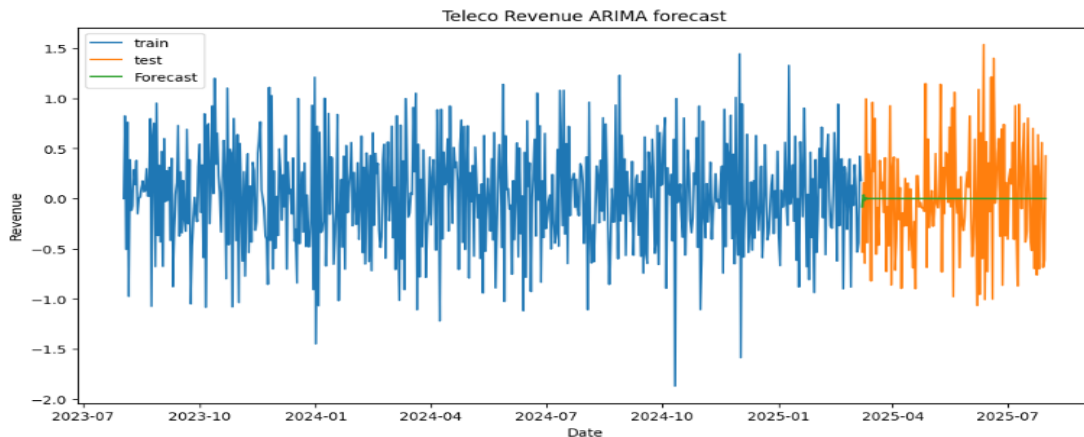
Warnings:

```
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

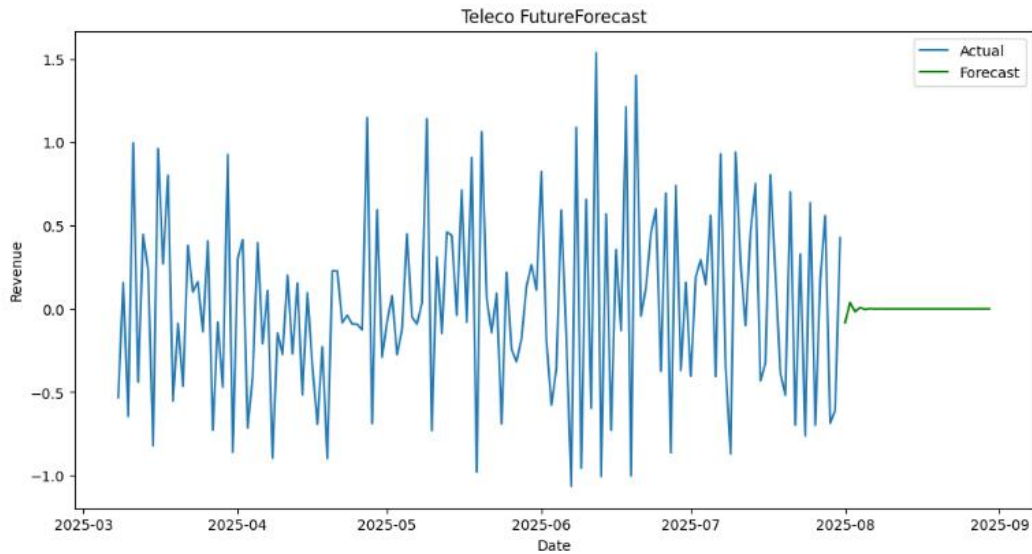
D3: FORECASTING USING ARIMA MODEL

```
#Using the Test data to forecast
forecast = Teleco_best_model.predict(n_periods=len(test_data))

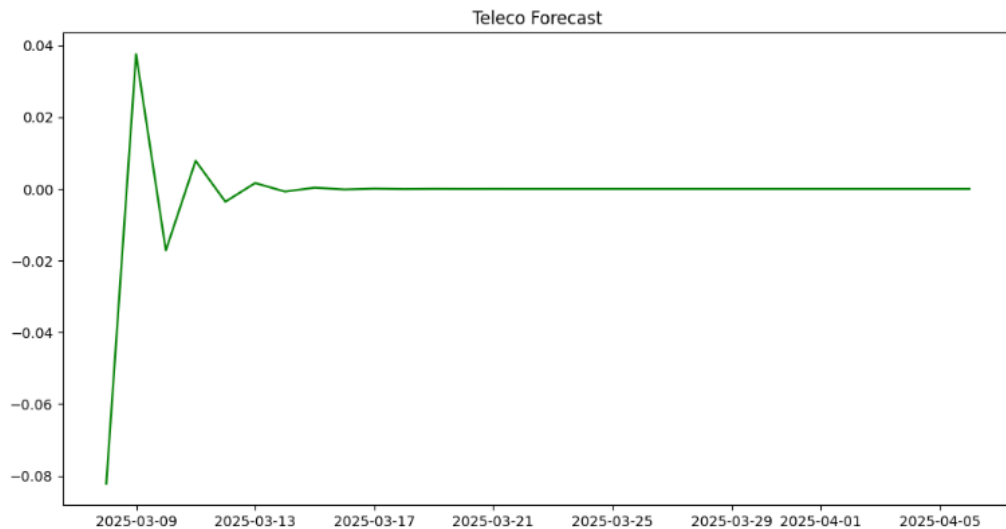
#forecast = Teleco_best_model.forecast(steps=len(test_data))
plt.figure(figsize=(12,6))
plt.plot(train_data.index, train_data['Revenue'], label='train')
plt.plot(test_data.index, test_data['Revenue'], label='test')
plt.plot(test_data.index, forecast, label='Forecast')
plt.title('Teleco Revenue ARIMA forecast')
plt.xlabel('Date')
plt.ylabel('Revenue')
plt.legend()
plt.show()
```



```
Teleco_future_forecast = Teleco_best_model.predict(n_periods=30)
plt.figure(figsize=(12,6))
plt.plot(test_data.index, test_data['Revenue'], label='Actual')
plt.plot(pd.date_range(start=Teleco_df.index[-1]+pd.Timedelta(days=1), periods=30), Teleco_future_forecast,
         label='Forecast', color='green')
plt.title('Teleco FutureForecast')
plt.xlabel('Date')
plt.ylabel('Revenue')
plt.legend()
plt.show()
```



```
#Forecasting for the next 30 days
future_forecast = Teleco_best_model.predict(n_periods=30)
plt.figure(figsize=(12,6))
plt.plot(future_forecast,color='green')
plt.title('Teleco Forecast')
plt.show()
```



```
#Validating the Models perfomance
mse = mean_squared_error(test_data, forecast)
mae = mean_absolute_error(test_data, forecast)
print(f"Mean Squared Error: {mse:.2f}, Mean Absolute Error: {mae:.2f}")
rmse = np.sqrt(mean_squared_error(test_data, forecast))
print("RMSE:", rmse)
```

Mean Squared Error: 0.32, Mean Absolute Error: 0.47
RMSE: 0.5700310582332536

```
forecast, confidence_interval = Teleco_best_model.predict(n_periods=30, return_conf_int=True)
Teleco_forecast_confidence_df= pd.DataFrame(
    {'Forecast':np.round(forecast,5),
     'Lower_ConfidenceInterval':np.round(confidence_interval[:,0],10),
     'Upper_ConfidenceInterval':np.round(confidence_interval[:,1],10)
    })
print(Teleco_forecast_confidence_df)
```

	Forecast	Lower_ConfidenceInterval	Upper_ConfidenceInterval
2025-03-08	-0.08220	-0.999676	0.835275
2025-03-09	0.03755	-0.971124	1.046227
2025-03-10	-0.01715	-1.043841	1.009532
2025-03-11	0.00784	-1.022569	1.038242
2025-03-12	-0.00358	-1.034760	1.027600
2025-03-13	0.00164	-1.029706	1.032977
2025-03-14	-0.00075	-1.032122	1.030628
2025-03-15	0.00034	-1.031041	1.031723
2025-03-16	-0.00016	-1.031539	1.031228
2025-03-17	0.00007	-1.031313	1.031455
2025-03-18	-0.00003	-1.031416	1.031351
2025-03-19	0.00001	-1.031369	1.031399
2025-03-20	-0.00001	-1.031391	1.031377
2025-03-21	0.00000	-1.031381	1.031387
2025-03-22	-0.00000	-1.031385	1.031382
2025-03-23	0.00000	-1.031383	1.031385
2025-03-24	-0.00000	-1.031384	1.031384
2025-03-25	0.00000	-1.031384	1.031384
2025-03-26	-0.00000	-1.031384	1.031384
2025-03-27	0.00000	-1.031384	1.031384
2025-03-28	-0.00000	-1.031384	1.031384
2025-03-29	0.00000	-1.031384	1.031384
2025-03-30	-0.00000	-1.031384	1.031384
2025-03-31	0.00000	-1.031384	1.031384
2025-04-01	-0.00000	-1.031384	1.031384
2025-04-02	0.00000	-1.031384	1.031384
2025-04-03	-0.00000	-1.031384	1.031384
2025-04-04	0.00000	-1.031384	1.031384
2025-04-05	-0.00000	-1.031384	1.031384
2025-04-06	0.00000	-1.031384	1.031384

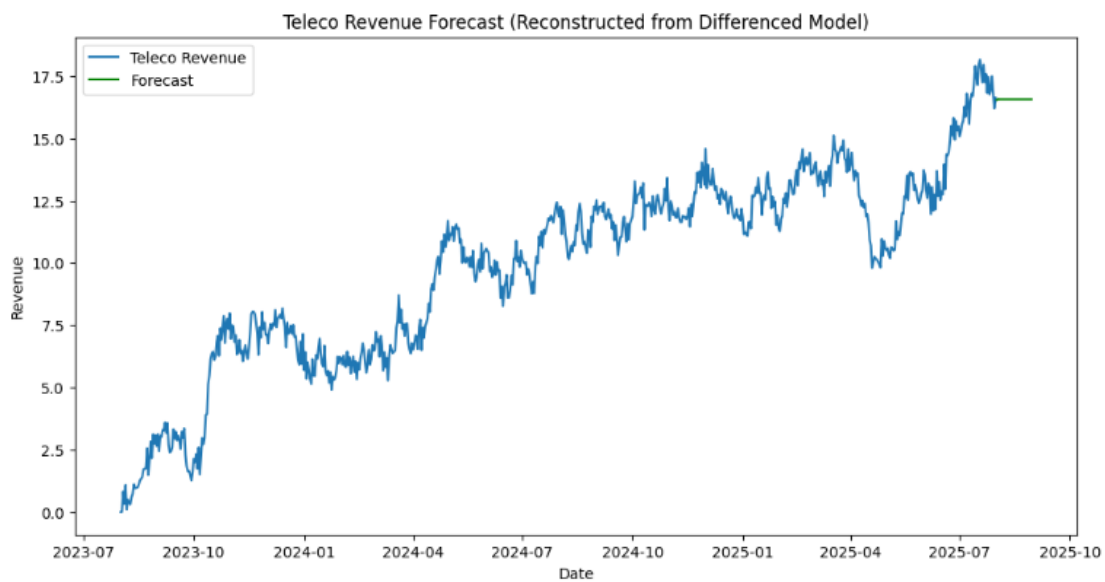
```
# Get the Last actual value before the forecast starts
last_actual = Teleco_df['Revenue'].iloc[-1]

# Reconstruct original forecast from differenced predictions
reconstructed_forecast = np.r_[last_actual, forecast].cumsum()[1:]

# Create a Series with future dates as index
future_dates = pd.date_range(start=Teleco_df.index[-1] + pd.Timedelta(days=1), periods=len(forecast))
forecast_series = pd.Series(reconstructed_forecast, index=future_dates)

# Plot the reconstructed forecast
plt.figure(figsize=(12,6))
plt.plot(Teleco_df.index, Teleco_df['Revenue'], label='Teleco Revenue')
plt.plot(future_dates, forecast_series, label='Forecast', color='green')
plt.title('Teleco Revenue Forecast (Reconstructed from Differenced Model)')
plt.xlabel('Date')
plt.ylabel('Revenue')
plt.legend()

plt.show()
```



```
# Reconstruct forecast and confidence intervals from differenced output
last_actual = Teleco_df['Revenue'].iloc[-1]

reconstructed_forecast = np.r_[last_actual, forecast].cumsum()[1:]
lower_bound = np.r_[last_actual, confidence_interval[:, 0]].cumsum()[1:]
upper_bound = np.r_[last_actual, confidence_interval[:, 1]].cumsum()[1:]

# Create DataFrame
Teleco_forecast_confidence_df = pd.DataFrame({
    'Forecast': np.round(reconstructed_forecast, 5),
    'Lower_ConfidenceInterval': np.round(lower_bound, 10),
    'Upper_ConfidenceInterval': np.round(upper_bound, 10)
})

print(Teleco_forecast_confidence_df)
```

	Forecast	Lower_ConfidenceInterval	Upper_ConfidenceInterval
0	16.53860	15.621122	17.456073
1	16.57615	14.649998	18.502300
2	16.55899	13.606158	19.511832
3	16.56683	12.583589	20.550074
4	16.56325	11.548829	21.577674
5	16.56489	10.519123	22.610650
6	16.56414	9.487001	23.641278
7	16.56448	8.455960	24.673001
8	16.56432	7.424421	25.704229
9	16.56440	6.393108	26.735684
10	16.56436	5.361692	27.767036
11	16.56438	4.330323	28.798434
12	16.56437	3.298932	29.829811
13	16.56437	2.267551	30.861198
14	16.56437	1.236166	31.892581
15	16.56437	0.204783	32.923965
16	16.56437	-0.826602	33.955349
17	16.56437	-1.857985	34.986733
18	16.56437	-2.889369	36.018117
19	16.56437	-3.920753	37.049501
20	16.56437	-4.952137	38.080885
21	16.56437	-5.983521	39.112269
22	16.56437	-7.014905	40.143653
23	16.56437	-8.046289	41.175037
24	16.56437	-9.077673	42.206420
25	16.56437	-10.109057	43.237804
26	16.56437	-11.140441	44.269188
27	16.56437	-12.171825	45.300572
28	16.56437	-13.203209	46.331956
29	16.56437	-14.234592	47.363340

D4: OUTPUT AND CALCULATIONS

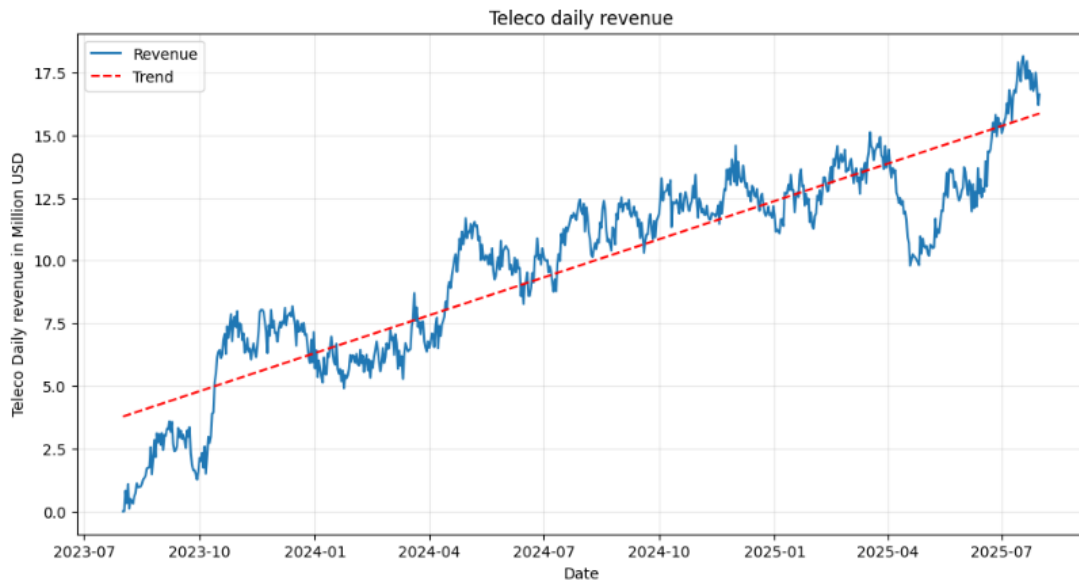
Provide the output and calculations of the analysis you performed.

```
: #LINE GRAPH VISUALIZATION
#Creating the initial time series graph to check for trend, seasonality,outlier
plt.figure(figsize=(12,6))
plt.plot(Teleco_df.index, Teleco_df, label="Revenue")

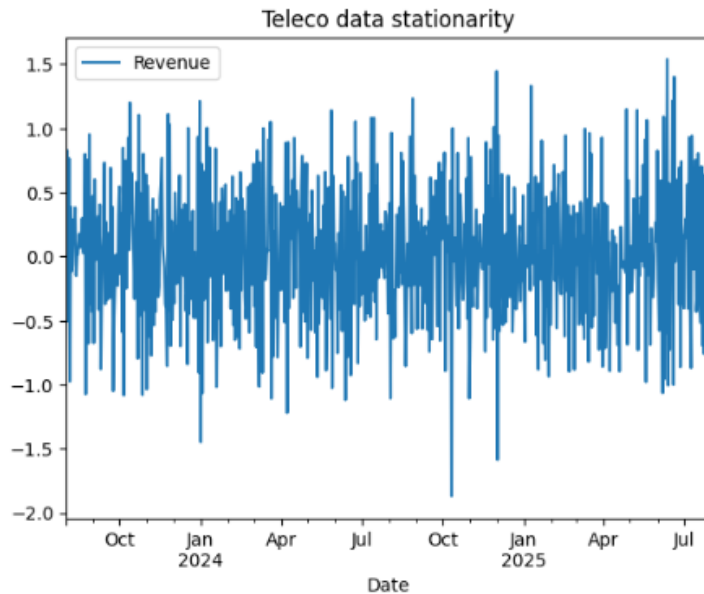
# Fitting the best line (polynomial) to the data
x = np.arange(len(Teleco_df['Revenue'].values))
y = Teleco_df['Revenue'].values

z = np.polyfit(x, y, 1)
p = np.poly1d(z)
plt.plot(Teleco_df.index, p(x), color='red',linestyle='--',label='Trend')

plt.title("Teleco daily revenue")
plt.ylabel("Teleco Daily revenue in Million USD")
plt.xlabel("Date")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```




```
#Data cleaning - Removing the trend/seasonality in the data
#(removing the trend from the teleco dataset to make it stationary like)
Teleco_df_diff=Teleco_df.diff().dropna()
Teleco_df_diff.plot()
plt.title('Teleco data stationarity')
plt.show()
```



```
#Stationarity test using the ADF test. Using the AIC autolag on the original data
Stationarity_Results = adfuller(Teleco_df['Revenue'].dropna())

labels = [
    "ADF Statistic",
    "p-value",
    "Used lags",
    "Number of observations",
    "Critical Values",
    "IC Best"
]

print("# Stationarity Test (ADF) Original data")
for label, value in zip(labels, Stationarity_Results):
    if label == "Critical Values":
        print(f"{label}:")
        for k, v in value.items():
            print(f"    {k}: {v:.4f}")
    else:
        print(f"{label}: {value}")

# Stationarity Test (ADF) Original data
ADF Statistic: -1.9246121573101838
p-value: 0.3205728150793964
Used lags: 1
Number of observations: 729
Critical Values:
    1%: -3.4394
    5%: -2.8655
    10%: -2.5689
IC Best: 965 0609576707513
```

```
#Rerunning the ADF Fuller on the differenced data
#Stationarity test using the ADF test
Stationarity_Results = adfuller(Teleco_df_diff['Revenue'].dropna())

labels = [
    "ADF Statistic",
    "p-value",
    "Used lags",
    "Number of observations",
    "Critical Values",
    "IC Best"
]

print("# Stationarity Test (ADF)-Differenced data")
for label, value in zip(labels, Stationarity_Results):
    if label == "Critical Values":
        print(f"{label}:")
        for k, v in value.items():
            print(f"    {k}: {v:.4f}")
    else:
        print(f"{label}: {value}")

# Stationarity Test (ADF)-Differenced data
ADF Statistic: -44.87452719387598
p-value: 0.0
Used lags: 0
Number of observations: 729
Critical Values:
    1%: -3.4394
    5%: -2.8655
    10%: -2.5689
IC Best: 965.5032159185916
```

```
#Splitting the data into training and testing data
train_data, test_data = train_test_split(Teleco_df_diff, test_size=0.2, random_state=42, shuffle=False)
train_data.head(5)
```

!]:

Revenue	
Date	
2023-08-02	0.000793
2023-08-03	0.824749
2023-08-04	-0.505210
2023-08-05	0.762222
2023-08-06	-0.974900

```
test_data.head()
```

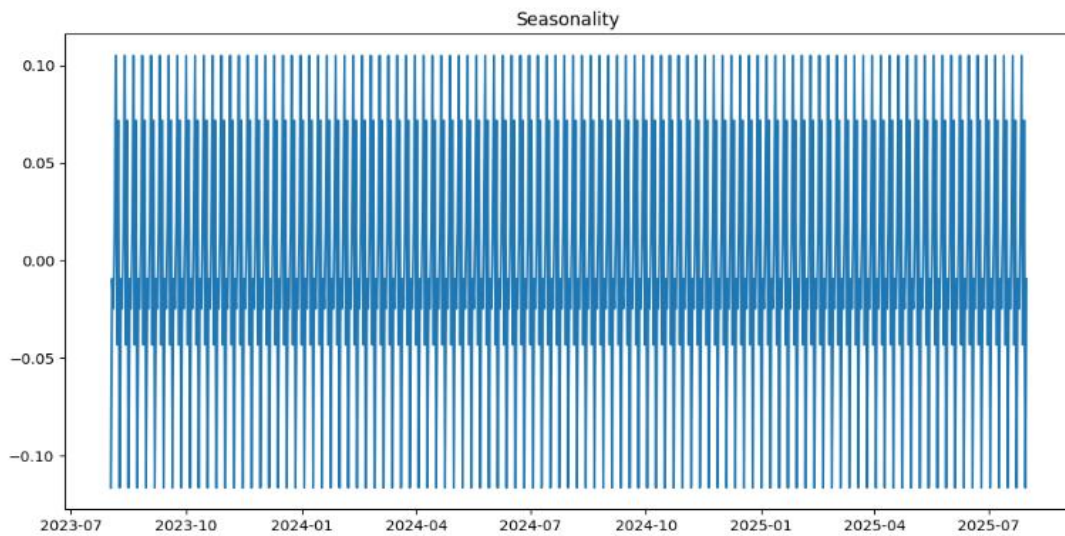
!]:

Revenue	
Date	
2025-03-08	-0.531923
2025-03-09	0.157387
2025-03-10	-0.644689
2025-03-11	0.995057
2025-03-12	-0.438775

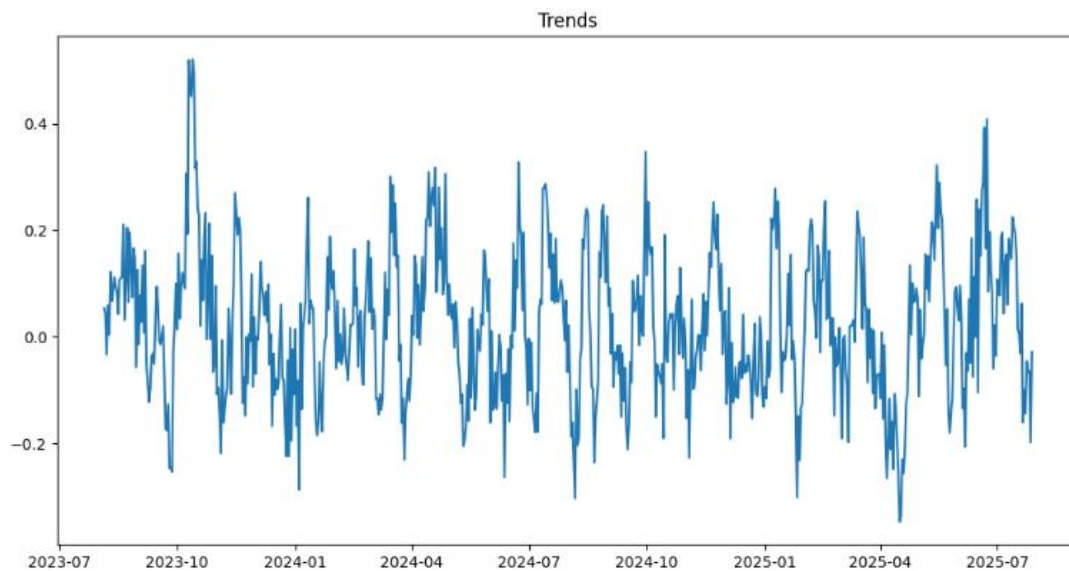
```
#Outputing the cleandata to csv
Teleco_df_diff.to_csv(CleanedFilePath)
train_data.to_csv(TrainedDataFilePath)
test_data.to_csv(TestDataFilePath)
```

```
#Seasonality
#the presence or lack of a seasonal component
decomposed_data= seasonal_decompose(Teleco_df_diff)
plt.figure(figsize=(12,6))
plt.title('Seasonality')
plt.plot(decomposed_data.seasonal)
```

[<matplotlib.lines.Line2D at 0x283ff89d050>]

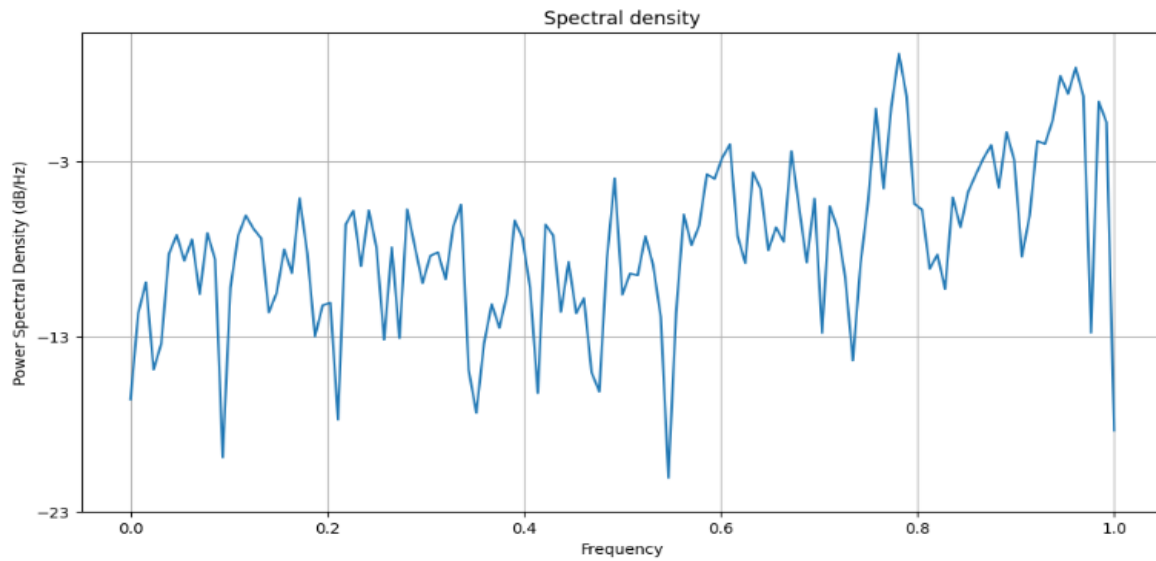


```
#Trends
plt.figure(figsize=(12,6))
plt.title('Trends')
plt.plot(decomposed_data.trend);
```



```
# the spectral density
plt.figure(figsize=(12,6))
plt.title('Spectral density')
plt.psd(x=Teleco_df_diff.Revenue)
plt.show()

frequencies,power= periodogram(x=Teleco_df_diff.Revenue)
Teleco_Spectral_df= pd.DataFrame({'Frequency':frequencies, 'Power':np.round(power,10)})
pd.set_option('display.max_rows', None)
print("Teleco Spectral dataframe")
print(Teleco_Spectral_df)
print (" ")
#REMOVING NULLS FROM FREQUENCY and power
Frequency = frequencies[frequencies > 0]
Power = power[frequencies > 0]
dominant_frequency = Frequency[np.argmax(Power)]
s = int(round(1 / dominant_frequency))
print(f"Dominant frequency: {dominant_frequency:.6f}")
print(f"Estimated seasonal period (s): {s}")
```



frequency

```
Teleco Spectral dataframe
  Frequency      Power
0    0.000000  0.000000
1    0.001370  0.107302
2    0.002740  0.179280
3    0.004110  0.177296
4    0.005479  0.012843
5    0.006849  0.408582
6    0.008219  0.235838
7    0.009589  0.201203
8    0.010959  0.090099
9    0.012329  0.272280
10   0.013699  0.057737
11   0.015068  0.123510
12   0.016438  0.058434
13   0.017808  0.572680
14   0.019178  0.775726
15   0.020548  0.496580
16   0.021918  0.385218
17   0.023288  0.429538
18   0.024658  0.501061
19   0.026027  0.179268
20   0.027397  0.026811
21   0.028767  0.155080
22   0.030137  0.227005
23   0.031507  0.398160
24   0.032877  0.444933
25   0.034247  0.292125
26   0.035616  0.021805
27   0.036986  0.135135
28   0.038356  0.149711
29   0.039726  0.609167
30   0.041096  0.369967
31   0.042466  0.168094
32   0.043836  0.153662
33   0.045205  0.044419
34   0.046575  0.041769
35   0.047945  0.164504
36   0.049315  0.423515
37   0.050685  0.058710
38   0.052055  0.098564
39   0.053425  0.136206
40   0.054795  0.378591
41   0.056164  0.046083
42   0.057534  0.016376
43   0.058904  0.576588
44   0.060274  0.299058
45   0.061644  0.340084
46   0.063014  0.022623
47   0.064384  0.048531
48   0.065753  0.393483
49   0.067123  0.261845
50   0.068493  0.120286
51   0.069863  0.029293
```

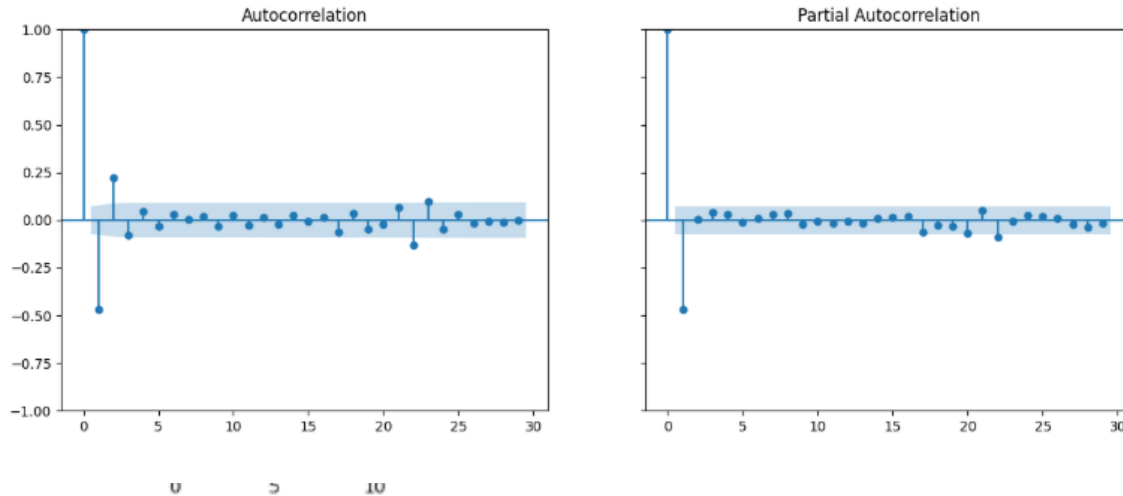
```
#autocorrelation function
print('Autocorrelation function')
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=[14,5], sharey=True)
plot_acf(Teleco_df_diff,ax=ax1)
plot_pacf(Teleco_df_diff,ax=ax2)
plt.show()

Acf_Value=acf(Teleco_df_diff)
Pacf_Value=pacf(Teleco_df_diff)

Teleco_lags=list(range(len(Acf_Value)))
Acf_Pacf_df= pd.DataFrame({'Lags':Teleco_lags, 'ACF':Acf_Value,'PACF':Pacf_Value})

print(Acf_Pacf_df)
```

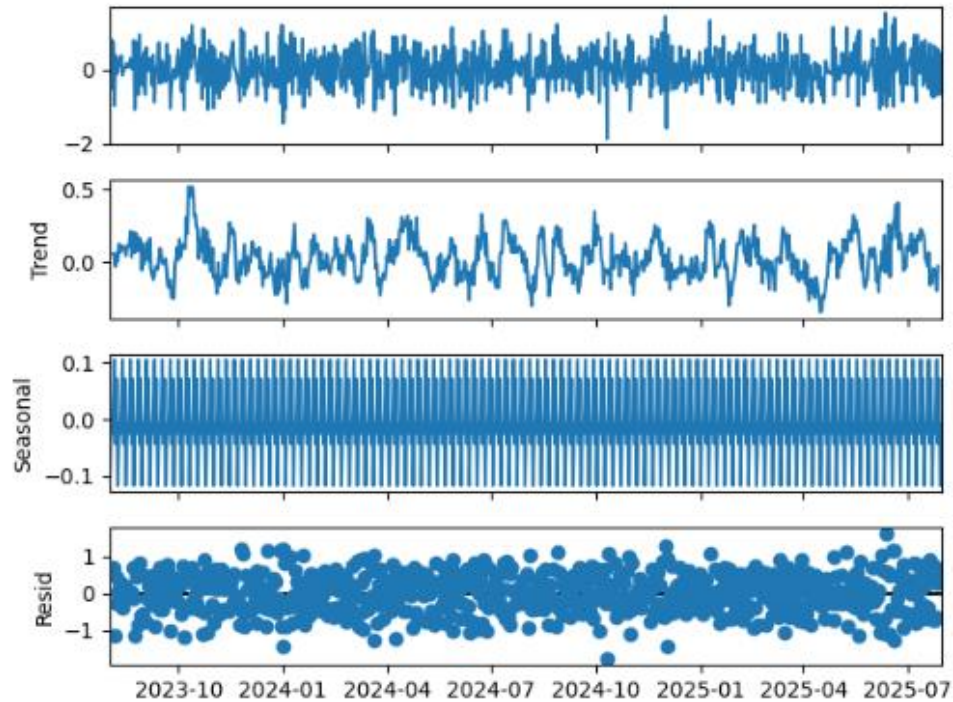
Autocorrelation function



	Lags	ACF	PACF
0	0	1.000000	1.000000
1	1	-0.469506	-0.470150
2	2	0.223874	0.004427
3	3	-0.076620	0.038797
4	4	0.047843	0.032467
5	5	-0.034508	-0.009443
6	6	0.029196	0.007075
7	7	0.004326	0.029870
8	8	0.017328	0.036432
9	9	-0.030830	-0.019598
10	10	0.023792	-0.006319
11	11	-0.027709	-0.018397
12	12	0.013641	-0.007650
13	13	-0.020448	-0.017175
14	14	0.022476	0.007463
15	15	-0.007156	0.013359
16	16	0.016461	0.021423
17	17	-0.064114	-0.065633
18	18	0.035488	-0.029407
19	19	-0.048303	-0.035548
20	20	-0.021149	-0.071277
21	21	0.068434	0.054827
22	22	-0.130043	-0.092892
23	23	0.095537	-0.008714
24	24	-0.046111	0.025421
25	25	0.028504	0.021075
26	26	-0.015962	0.007968
27	27	-0.008878	-0.025269
28	28	-0.012601	-0.036867

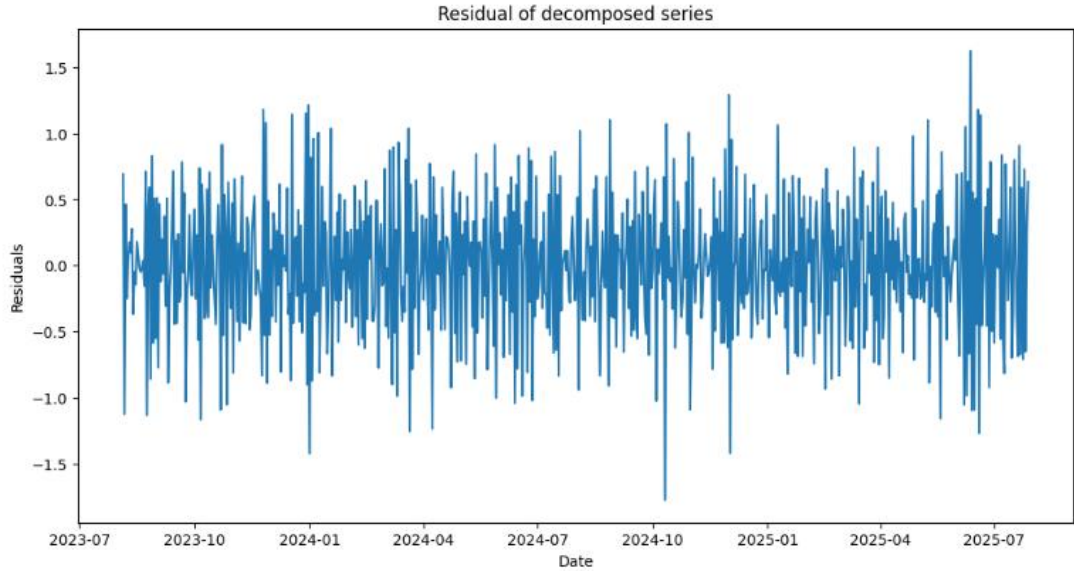
```
#the decomposed time series  
plt.figure(figsize=(18,10))  
decomposed_data.plot()  
plt.show()
```

<Figure size 1800x1000 with 0 Axes>




```
#confirmation of the Lack of trends in the residuals of the decomposed series
plt.figure(figsize=(12,6))
plt.title('Residual of decomposed series')
plt.xlabel("Date")
plt.ylabel("Residuals")
plt.plot(decomposed_data.resid)

: [ <matplotlib.lines.Line2D at 0x2838205fd50>]
```



```
#Creating the initial model
Initial_model = ARIMA(train_data, order=(1,1,1))
Initial_model_fit = Initial_model.fit()
print(Initial_model_fit.summary())
```

```
SARIMAX Results
=====
Dep. Variable:          Revenue    No. Observations:          584
Model:                 ARIMA(1, 1, 1)    Log Likelihood            -387.352
Date:                 Tue, 16 Sep 2025    AIC                       780.703
Time:                 19:49:11           BIC                       793.808
Sample:              08-02-2023         HQIC                      785.811
                  - 03-07-2025
Covariance Type:      opg
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ar.L1         -0.4587     0.037    -12.556     0.000     -0.530    -0.387
ma.L1         -0.9997     0.098    -10.191     0.000     -1.192    -0.807
sigma2         0.2184     0.024     9.004     0.000     0.171     0.266
=====
Ljung-Box (L1) (Q):                0.01    Jarque-Bera (JB):                1.93
Prob(Q):                          0.91    Prob(JB):                  0.38
Heteroskedasticity (H):            0.96    Skew:                      -0.08
Prob(H) (two-sided):              0.75    Kurtosis:                  2.76
=====
```

Warnings:

```
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```



```
#Determining the best ARIMA model
Teleco_best_model = auto_arima(train_data, start_p=0, start_q=0, max_p=3, max_q=3, d=None,
                              seasonal=True, stepwise=True, information_criterion='bic', trace=True)
print(Teleco_best_model.summary())
```

Performing stepwise search to minimize bic

ARIMA(0,0,0)(0,0,0)[0] intercept	:	BIC=919.529, Time=0.05 sec
ARIMA(1,0,0)(0,0,0)[0] intercept	:	BIC=787.002, Time=0.05 sec
ARIMA(0,0,1)(0,0,0)[0] intercept	:	BIC=812.574, Time=0.06 sec
ARIMA(0,0,0)(0,0,0)[0]	:	BIC=914.318, Time=0.03 sec
ARIMA(2,0,0)(0,0,0)[0] intercept	:	BIC=793.366, Time=0.06 sec
ARIMA(1,0,1)(0,0,0)[0] intercept	:	BIC=793.367, Time=0.08 sec
ARIMA(2,0,1)(0,0,0)[0] intercept	:	BIC=799.543, Time=0.58 sec
ARIMA(1,0,0)(0,0,0)[0]	:	BIC=783.730, Time=0.02 sec
ARIMA(2,0,0)(0,0,0)[0]	:	BIC=790.098, Time=0.04 sec
ARIMA(1,0,1)(0,0,0)[0]	:	BIC=790.099, Time=0.05 sec
ARIMA(0,0,1)(0,0,0)[0]	:	BIC=809.839, Time=0.05 sec
ARIMA(2,0,1)(0,0,0)[0]	:	BIC=795.977, Time=0.13 sec

Best model: ARIMA(1,0,0)(0,0,0)[0]
Total fit time: 1.201 seconds

SARIMAX Results

```
=====
Dep. Variable:          y      No. Observations:          584
Model:                SARIMAX(1, 0, 0)      Log Likelihood:      -385.495
Date:                Tue, 16 Sep 2025      AIC:                774.990
Time:                19:47:57      BIC:                783.730
Sample:                08-02-2023      HQIC:               778.397
                    - 03-07-2025

Covariance Type:      opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	-0.4568	0.036	-12.605	0.000	-0.528	-0.386
sigma2	0.2191	0.014	15.965	0.000	0.192	0.246

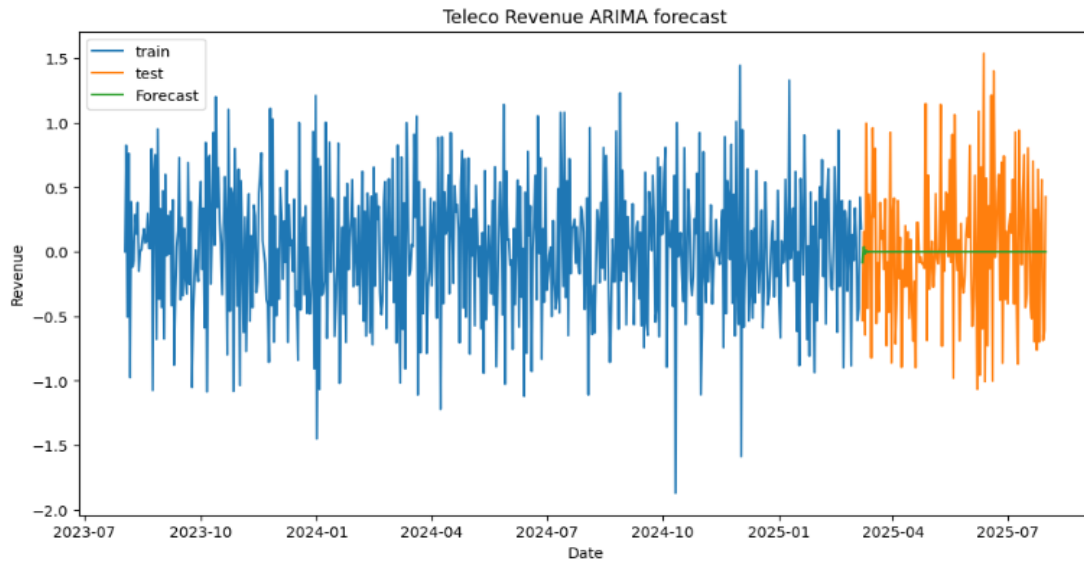
```
=====
Ljung-Box (L1) (Q):                0.02      Jarque-Bera (JB):                1.85
Prob(Q):                          0.90      Prob(JB):                0.40
Heteroskedasticity (H):            0.96      Skew:                    -0.08
Prob(H) (two-sided):              0.79      Kurtosis:                2.77
=====
```

Warnings:

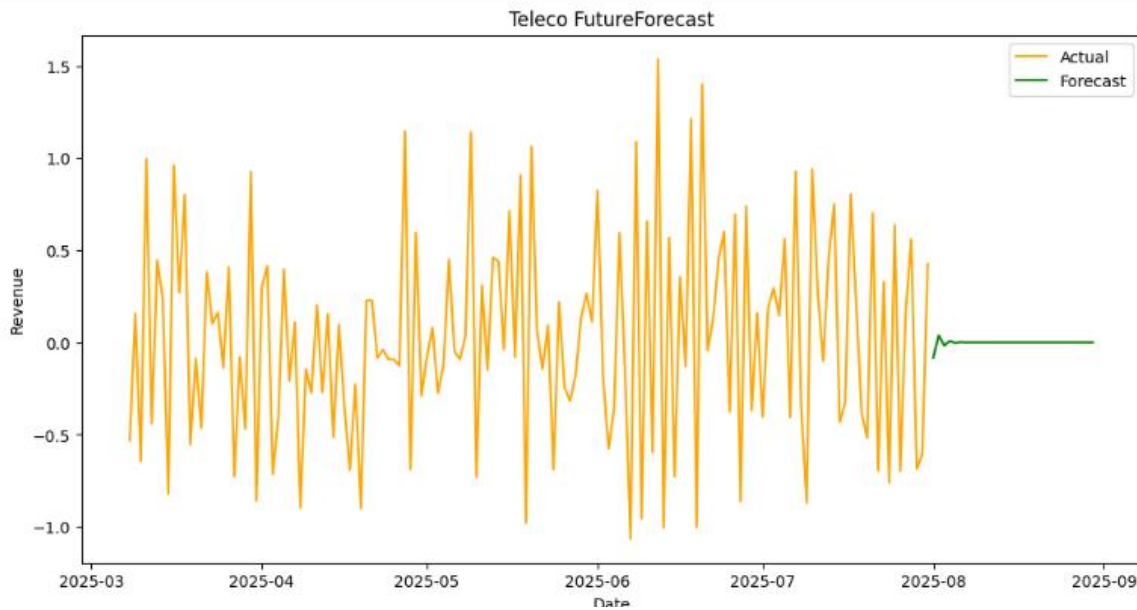
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```
#Using the Test data to forecast
forecast = Teleco_best_model.predict(n_periods=len(test_data))

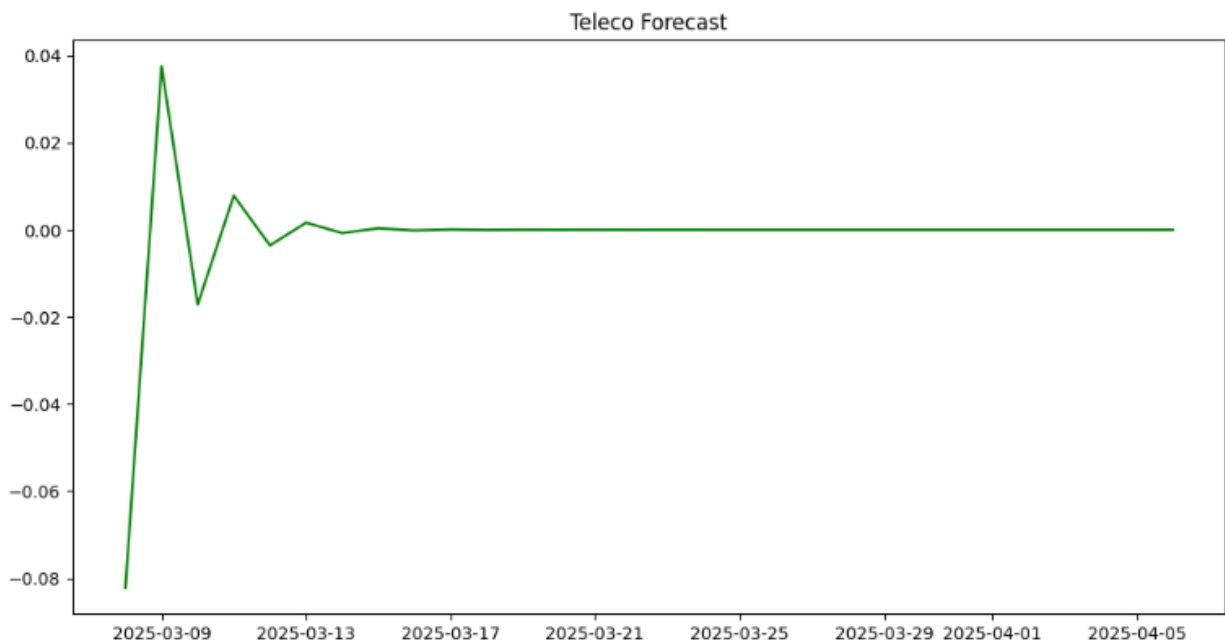
#forecast = Teleco_best_model.forecast(steps=len(test_data))
plt.figure(figsize=(12,6))
plt.plot(train_data.index, train_data['Revenue'], label='train')
plt.plot(test_data.index, test_data['Revenue'], label='test')
plt.plot(test_data.index, forecast, label='Forecast')
plt.title('Teleco Revenue ARIMA forecast')
plt.xlabel('Date')
plt.ylabel('Revenue')
plt.legend()
plt.show()
```



```
Teleco_future_forecast = Teleco_best_model.predict(n_periods=30)
plt.figure(figsize=(12,6))
plt.plot(test_data.index, test_data['Revenue'], label='Actual',color='orange')
plt.plot(pd.date_range(start=Teleco_df.index[-1]+pd.Timedelta(days=1), periods=30), Teleco_future_forecast,
        label='Forecast', color='green')
plt.title('Teleco FutureForecast')
plt.xlabel('Date')
plt.ylabel('Revenue')
plt.legend()
plt.show()
```



```
#Forecasting for the next 30 days
future_forecast = Teleco_best_model.predict(n_periods=30)
plt.figure(figsize=(12,6))
plt.plot(future_forecast,color='green')
plt.title('Teleco Forecast')
plt.show()
```



```
#Validating the Models performance
mse = mean_squared_error(test_data, forecast)
mae = mean_absolute_error(test_data, forecast)
print(f"Mean Squared Error: {mse:.2f}, Mean Absolute Error: {mae:.2f}")
rmse = np.sqrt(mean_squared_error(test_data, forecast))
print("RMSE:", rmse)
```

Mean Squared Error: 0.32, Mean Absolute Error: 0.47
RMSE: 0.5700310582332536

```
forecast, confidence_interval = Teleco_best_model.predict(n_periods=30, return_conf_int=True)
Teleco_forecast_confidence_df= pd.DataFrame(
    {'Forecast':np.round(forecast,5),
     'Lower_ConfidenceInterval':np.round(confidence_interval[:,0],10),
     'Upper_ConfidenceInterval':np.round(confidence_interval[:,1],10)
    })
print(Teleco_forecast_confidence_df)
```

	Forecast	Lower_ConfidenceInterval	Upper_ConfidenceInterval
2025-03-08	-0.08220	-0.999676	0.835275
2025-03-09	0.03755	-0.971124	1.046227
2025-03-10	-0.01715	-1.043841	1.009532
2025-03-11	0.00784	-1.022569	1.038242
2025-03-12	-0.00358	-1.034760	1.027600
2025-03-13	0.00164	-1.029706	1.032977
2025-03-14	-0.00075	-1.032122	1.030628
2025-03-15	0.00034	-1.031041	1.031723
2025-03-16	-0.00016	-1.031539	1.031228
2025-03-17	0.00007	-1.031313	1.031455
2025-03-18	-0.00003	-1.031416	1.031351
2025-03-19	0.00001	-1.031369	1.031399
2025-03-20	-0.00001	-1.031391	1.031377
2025-03-21	0.00000	-1.031381	1.031387
2025-03-22	-0.00000	-1.031385	1.031382
2025-03-23	0.00000	-1.031383	1.031385
2025-03-24	-0.00000	-1.031384	1.031384
2025-03-25	0.00000	-1.031384	1.031384
2025-03-26	-0.00000	-1.031384	1.031384
2025-03-27	0.00000	-1.031384	1.031384
2025-03-28	-0.00000	-1.031384	1.031384
2025-03-29	0.00000	-1.031384	1.031384
2025-03-30	-0.00000	-1.031384	1.031384
2025-03-31	0.00000	-1.031384	1.031384
2025-04-01	-0.00000	-1.031384	1.031384
2025-04-02	0.00000	-1.031384	1.031384
2025-04-03	-0.00000	-1.031384	1.031384
2025-04-04	0.00000	-1.031384	1.031384
2025-04-05	-0.00000	-1.031384	1.031384
2025-04-06	0.00000	-1.031384	1.031384

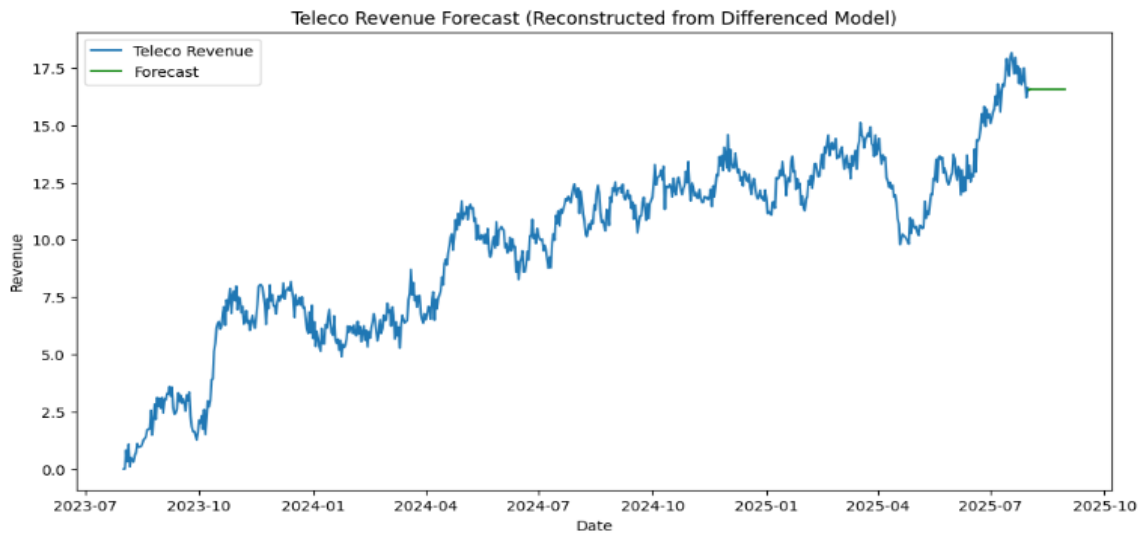
```
#using the model to predict future revenue values
# Get the last actual value before the forecast starts
last_actual = Teleco_df['Revenue'].iloc[-1]

# Reconstruct original forecast from differenced predictions
reconstructed_forecast = np.r_[last_actual, forecast].cumsum()[1:]

# Create a Series with future dates as index
future_dates = pd.date_range(start=Teleco_df.index[-1] + pd.Timedelta(days=1), periods=len(forecast))
forecast_series = pd.Series(reconstructed_forecast, index=future_dates)

# Plot the reconstructed forecast
plt.figure(figsize=(12,6))
plt.plot(Teleco_df.index, Teleco_df['Revenue'], label='Teleco Revenue')
plt.plot(future_dates, forecast_series, label='Forecast', color='green')
plt.title('Teleco Revenue Forecast (Reconstructed from Differenced Model)')
plt.xlabel('Date')
plt.ylabel('Revenue')
plt.legend()

plt.show()
```



```
# Reconstruct forecast and confidence intervals from differenced output
last_actual = Teleco_df['Revenue'].iloc[-1]

reconstructed_forecast = np.r_[last_actual, forecast].cumsum()[1:]
lower_bound = np.r_[last_actual, confidence_interval[:, 0]].cumsum()[1:]
upper_bound = np.r_[last_actual, confidence_interval[:, 1]].cumsum()[1:]

# Create DataFrame
Teleco_forecast_confidence_df = pd.DataFrame({
    'Forecast': np.round(reconstructed_forecast, 5),
    'Lower_ConfidenceInterval': np.round(lower_bound, 10),
    'Upper_ConfidenceInterval': np.round(upper_bound, 10)
})

print(Teleco_forecast_confidence_df)
```

	Forecast	Lower_ConfidenceInterval	Upper_ConfidenceInterval
0	16.53860	15.621122	17.456073
1	16.57615	14.649998	18.502300
2	16.55899	13.606158	19.511832
3	16.56683	12.583589	20.550074
4	16.56325	11.548829	21.577674
5	16.56489	10.519123	22.610650
6	16.56414	9.487001	23.641278
7	16.56448	8.455960	24.673001
8	16.56432	7.424421	25.704229
9	16.56440	6.393108	26.735684
10	16.56436	5.361692	27.767036
11	16.56438	4.330323	28.798434
12	16.56437	3.298932	29.829811
13	16.56437	2.267551	30.861198
14	16.56437	1.236166	31.892581
15	16.56437	0.204783	32.923965
16	16.56437	-0.826602	33.955349
17	16.56437	-1.857985	34.986733
18	16.56437	-2.889369	36.018117
19	16.56437	-3.920753	37.049501
20	16.56437	-4.952137	38.080885
21	16.56437	-5.983521	39.112269
22	16.56437	-7.014905	40.143653
23	16.56437	-8.046289	41.175037
24	16.56437	-9.077673	42.206420
25	16.56437	-10.109057	43.237804
26	16.56437	-11.140441	44.269188
27	16.56437	-12.171825	45.300572
28	16.56437	-13.203209	46.331956
29	16.56437	-14.234592	47.363340

D5: CODE

Jupyter notebook code – Executable code

Please modify the local file path in the code

Local_path = 'C:/Users/beatr/OneDrive/Desktop/WGU/D213/Task1/Data/'



BeatriceMwangi_D2
13 Advanced Data A

PDF output



BeatriceMwangi_D2
13 Advanced Data A

E1: RESULTS

Discuss the results of your data analysis, including the following points:

- the selection of an ARIMA model

In the selection of the ARIMA model to use, `auto_arima` from the `pmdarima` library was used to dynamically determine the best model.

Initial manual assessment of the ARIMA model, using the $AR = 1$, $I = 1$, and $MA = 1$, performed poorly and had higher AIC(780.70) and BIC (793.81), indicating over differencing, and the $MA(1)$ was insignificant.

The dynamic ARIMA model (1,0,0) produced a better AIC of 774.99 and BIC of 783.73, confirming that the model is more parsimonious and explains the data well using the fewest parameters and avoids over-fitting of the data.

```
]: M #Determining the best ARIMA model
Teleco_best_model = auto_arma(train_data, start_p=0, start_q=0, max_p=3, max_q=3, d=None,
                              seasonal=True, stepwise=True, information_criterion='bic', trace=True)
print(Teleco_best_model.summary())
```

```
Performing stepwise search to minimize bic
ARIMA(0,0,0)(0,0,0)[0] intercept : BIC=919.529, Time=0.06 sec
ARIMA(1,0,0)(0,0,0)[0] intercept : BIC=787.002, Time=0.05 sec
ARIMA(0,0,1)(0,0,0)[0] intercept : BIC=812.574, Time=0.08 sec
ARIMA(0,0,0)(0,0,0)[0] : BIC=914.318, Time=0.04 sec
ARIMA(2,0,0)(0,0,0)[0] intercept : BIC=793.366, Time=0.07 sec
ARIMA(1,0,1)(0,0,0)[0] intercept : BIC=793.367, Time=0.08 sec
ARIMA(2,0,1)(0,0,0)[0] intercept : BIC=799.543, Time=0.49 sec
ARIMA(1,0,0)(0,0,0)[0] : BIC=783.730, Time=0.03 sec
ARIMA(2,0,0)(0,0,0)[0] : BIC=790.098, Time=0.03 sec
ARIMA(1,0,1)(0,0,0)[0] : BIC=790.099, Time=0.05 sec
ARIMA(0,0,1)(0,0,0)[0] : BIC=809.839, Time=0.06 sec
ARIMA(2,0,1)(0,0,0)[0] : BIC=795.977, Time=0.15 sec
```

```
Best model: ARIMA(1,0,0)(0,0,0)[0]
Total fit time: 1.179 seconds
```

SARIMAX Results

```
=====
Dep. Variable:          y      No. Observations:          584
Model:                SARIMAX(1, 0, 0)      Log Likelihood      -385.495
Date:                Tue, 16 Sep 2025      AIC                  774.990
Time:                18:20:19      BIC                  783.730
Sample:              08-02-2023      HQIC                 778.397
                  - 03-07-2025
```

```
Covariance Type:      opg
```

```
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ar.L1          -0.4568         0.036     -12.605      0.000      -0.528      -0.386
sigma2          0.2191         0.014      15.965      0.000         0.192         0.246
=====
```

```
Ljung-Box (L1) (Q):          0.02      Jarque-Bera (JB):          1.85
Prob(Q):                  0.90      Prob(JB):              0.40
Heteroskedasticity (H):      0.96      Skew:                 -0.08
Prob(H) (two-sided):        0.79      Kurtosis:             2.77
=====
```

Warnings:

```
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```


- the prediction interval of the forecast

The ARIMA model(1,0,0) provided a forecast with a 97.5% confidence interval, meaning that for the forecasted revenue, the model accurately predicts that 97.5 % of the time the revenue value will be within that range.

```

forecast, confidence_interval = Teleco_best_model.predict(n_periods=30, return_conf_int=True)
Teleco_forecast_confidence_df= pd.DataFrame(
    {'Forecast':np.round(forecast,5),
     'Lower_ConfidenceInterval':np.round(confidence_interval[:,0],10),
     'Upper_ConfidenceInterval':np.round(confidence_interval[:,1],10)}
)
print(Teleco_forecast_confidence_df)

```

	Forecast	Lower_ConfidenceInterval	Upper_ConfidenceInterval
2025-03-08	-0.08220	-0.999676	0.835275
2025-03-09	0.03755	-0.971124	1.046227
2025-03-10	-0.01715	-1.043841	1.009532
2025-03-11	0.00784	-1.022569	1.038242
2025-03-12	-0.00358	-1.034760	1.027600
2025-03-13	0.00164	-1.029706	1.032977
2025-03-14	-0.00075	-1.032122	1.030628
2025-03-15	0.00034	-1.031041	1.031723
2025-03-16	-0.00016	-1.031539	1.031228
2025-03-17	0.00007	-1.031313	1.031455
2025-03-18	-0.00003	-1.031416	1.031351
2025-03-19	0.00001	-1.031369	1.031399
2025-03-20	-0.00001	-1.031391	1.031377
2025-03-21	0.00000	-1.031381	1.031387
2025-03-22	-0.00000	-1.031385	1.031382
2025-03-23	0.00000	-1.031383	1.031385
2025-03-24	-0.00000	-1.031384	1.031384
2025-03-25	0.00000	-1.031384	1.031384
2025-03-26	-0.00000	-1.031384	1.031384
2025-03-27	0.00000	-1.031384	1.031384
2025-03-28	-0.00000	-1.031384	1.031384
2025-03-29	0.00000	-1.031384	1.031384
2025-03-30	-0.00000	-1.031384	1.031384
2025-03-31	0.00000	-1.031384	1.031384
2025-04-01	-0.00000	-1.031384	1.031384
2025-04-02	0.00000	-1.031384	1.031384
2025-04-03	-0.00000	-1.031384	1.031384
2025-04-04	0.00000	-1.031384	1.031384
2025-04-05	-0.00000	-1.031384	1.031384
2025-04-06	0.00000	-1.031384	1.031384

```
# Reconstruct forecast and confidence intervals from differenced output
last_actual = Teleco_df['Revenue'].iloc[-1]

reconstructed_forecast = np.r_[last_actual, forecast].cumsum()[1:]
lower_bound = np.r_[last_actual, confidence_interval[:, 0]].cumsum()[1:]
upper_bound = np.r_[last_actual, confidence_interval[:, 1]].cumsum()[1:]

# Create DataFrame
Teleco_forecast_confidence_df = pd.DataFrame({
    'Forecast': np.round(reconstructed_forecast, 5),
    'Lower_ConfidenceInterval': np.round(lower_bound, 10),
    'Upper_ConfidenceInterval': np.round(upper_bound, 10)
})

print(Teleco_forecast_confidence_df)
```

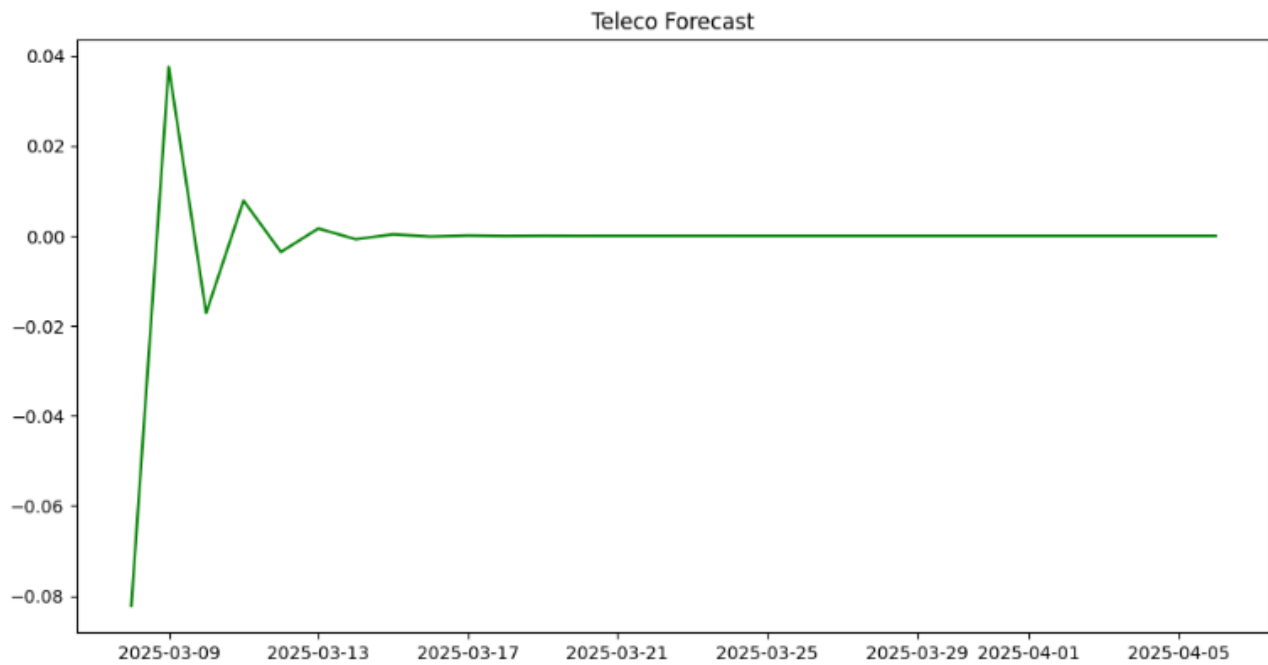
	Forecast	Lower_ConfidenceInterval	Upper_ConfidenceInterval
0	16.53860	15.621122	17.456073
1	16.57615	14.649998	18.502300
2	16.55899	13.606158	19.511832
3	16.56683	12.583589	20.550074
4	16.56325	11.548829	21.577674
5	16.56489	10.519123	22.610650
6	16.56414	9.487001	23.641278
7	16.56448	8.455960	24.673001
8	16.56432	7.424421	25.704229
9	16.56440	6.393108	26.735684
10	16.56436	5.361692	27.767036
11	16.56438	4.330323	28.798434
12	16.56437	3.298932	29.829811
13	16.56437	2.267551	30.861198
14	16.56437	1.236166	31.892581
15	16.56437	0.204783	32.923965
16	16.56437	-0.826602	33.955349
17	16.56437	-1.857985	34.986733
18	16.56437	-2.889369	36.018117
19	16.56437	-3.920753	37.049501
20	16.56437	-4.952137	38.080885
21	16.56437	-5.983521	39.112269
22	16.56437	-7.014905	40.143653
23	16.56437	-8.046289	41.175037
24	16.56437	-9.077673	42.206420
25	16.56437	-10.109057	43.237804
26	16.56437	-11.140441	44.269188
27	16.56437	-12.171825	45.300572
28	16.56437	-13.203209	46.331956
29	16.56437	-14.234592	47.363340

- a justification of the forecast length

In choosing the forecasting duration, I choose to forecast for 30 days since the data is noisy and lacks seasonality. We see the forecast prediction for the first 12 days, and thereafter, the forecast flattens out. Therefore, for this ARIMA model, the best forecast duration for the teleco data

would be 7 days. This is also in line with the ACF values, where past lag 2, the ACF values are low and within the confidence interval. Also observed is that the confidence interval widens as the number of days increases.

```
#Forecasting for the next 30 days
future_forecast = Teleco_best_model.predict(n_periods=30)
plt.figure(figsize=(12,6))
plt.plot(future_forecast,color='green')
plt.title('Teleco Forecast')
plt.show()
```



```

forecast, confidence_interval = Teleco_best_model.predict(n_periods=30, return_conf_int=True)
Teleco_forecast_confidence_df= pd.DataFrame(
    {'Forecast':np.round(forecast,5),
     'Lower_ConfidenceInterval':np.round(confidence_interval[:,0],10),
     'Upper_ConfidenceInterval':np.round(confidence_interval[:,1],10)
    })
print(Teleco_forecast_confidence_df)

```

	Forecast	Lower_ConfidenceInterval	Upper_ConfidenceInterval
2025-03-08	-0.08220	-0.999676	0.835275
2025-03-09	0.03755	-0.971124	1.046227
2025-03-10	-0.01715	-1.043841	1.009532
2025-03-11	0.00784	-1.022569	1.038242
2025-03-12	-0.00358	-1.034760	1.027600
2025-03-13	0.00164	-1.029706	1.032977
2025-03-14	-0.00075	-1.032122	1.030628
2025-03-15	0.00034	-1.031041	1.031723
2025-03-16	-0.00016	-1.031539	1.031228
2025-03-17	0.00007	-1.031313	1.031455
2025-03-18	-0.00003	-1.031416	1.031351
2025-03-19	0.00001	-1.031369	1.031399
2025-03-20	-0.00001	-1.031391	1.031377
2025-03-21	0.00000	-1.031381	1.031387
2025-03-22	-0.00000	-1.031385	1.031382
2025-03-23	0.00000	-1.031383	1.031385
2025-03-24	-0.00000	-1.031384	1.031384
2025-03-25	0.00000	-1.031384	1.031384
2025-03-26	-0.00000	-1.031384	1.031384
2025-03-27	0.00000	-1.031384	1.031384
2025-03-28	-0.00000	-1.031384	1.031384
2025-03-29	0.00000	-1.031384	1.031384
2025-03-30	-0.00000	-1.031384	1.031384
2025-03-31	0.00000	-1.031384	1.031384
2025-04-01	-0.00000	-1.031384	1.031384
2025-04-02	0.00000	-1.031384	1.031384
2025-04-03	-0.00000	-1.031384	1.031384
2025-04-04	0.00000	-1.031384	1.031384
2025-04-05	-0.00000	-1.031384	1.031384
2025-04-06	0.00000	-1.031384	1.031384

```
# Reconstruct forecast and confidence intervals from differenced output
last_actual = Teleco_df['Revenue'].iloc[-1]

reconstructed_forecast = np.r_[last_actual, forecast].cumsum()[1:]
lower_bound = np.r_[last_actual, confidence_interval[:, 0]].cumsum()[1:]
upper_bound = np.r_[last_actual, confidence_interval[:, 1]].cumsum()[1:]

# Create DataFrame
Teleco_forecast_confidence_df = pd.DataFrame({
    'Forecast': np.round(reconstructed_forecast, 5),
    'Lower_ConfidenceInterval': np.round(lower_bound, 10),
    'Upper_ConfidenceInterval': np.round(upper_bound, 10)
})

print(Teleco_forecast_confidence_df)
```

	Forecast	Lower_ConfidenceInterval	Upper_ConfidenceInterval
0	16.53860	15.621122	17.456073
1	16.57615	14.649998	18.502300
2	16.55899	13.606158	19.511832
3	16.56683	12.583589	20.550074
4	16.56325	11.548829	21.577674
5	16.56489	10.519123	22.610650
6	16.56414	9.487001	23.641278
7	16.56448	8.455960	24.673001
8	16.56432	7.424421	25.704229
9	16.56440	6.393108	26.735684
10	16.56436	5.361692	27.767036
11	16.56438	4.330323	28.798434
12	16.56437	3.298932	29.829811
13	16.56437	2.267551	30.861198
14	16.56437	1.236166	31.892581
15	16.56437	0.204783	32.923965
16	16.56437	-0.826602	33.955349
17	16.56437	-1.857985	34.986733
18	16.56437	-2.889369	36.018117
19	16.56437	-3.920753	37.049501
20	16.56437	-4.952137	38.080885
21	16.56437	-5.983521	39.112269
22	16.56437	-7.014905	40.143653
23	16.56437	-8.046289	41.175037
24	16.56437	-9.077673	42.206420
25	16.56437	-10.109057	43.237804
26	16.56437	-11.140441	44.269188
27	16.56437	-12.171825	45.300572
28	16.56437	-13.203209	46.331956
29	16.56437	-14.234592	47.363340

- the model evaluation procedure and error metric

The Mean Squared Error (MSE), the Mean Absolute Error(MAE), and the Root Mean Squared Error (RMSE) were used to evaluate the model's performance. The MSE measures the average squared difference between predicted and actual values; on the other hand, MAE is a metric that measures the average magnitude of errors without squaring them, while RMSE is the square root of MSE and represents the standard deviation of the residuals. In assessing the accuracy of the model, the MSE value was 0.32, the MAE was 0.47, and the RMSE was 0.57, showing that the model fits the data well.

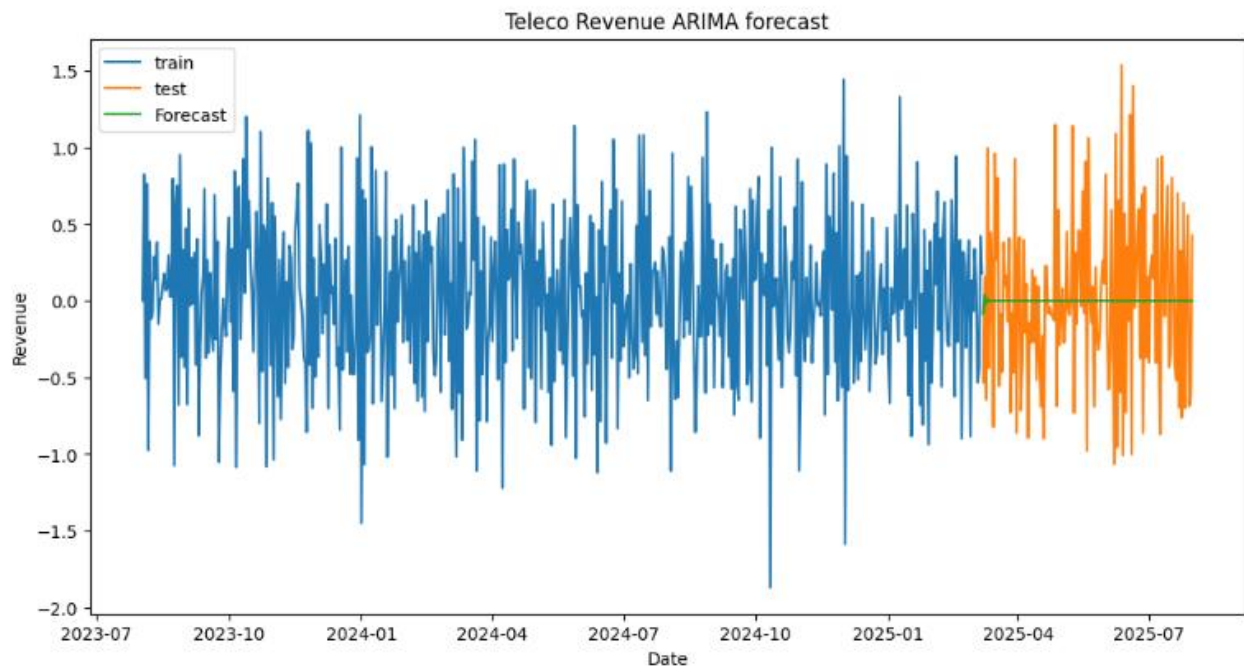
```
#Validating the Models performance
mse = mean_squared_error(test_data, forecast)
mae = mean_absolute_error(test_data, forecast)
print(f"Mean Squared Error: {mse:.2f}, Mean Absolute Error: {mae:.2f}")
rmse = np.sqrt(mean_squared_error(test_data, forecast))
print("RMSE:", rmse)
```

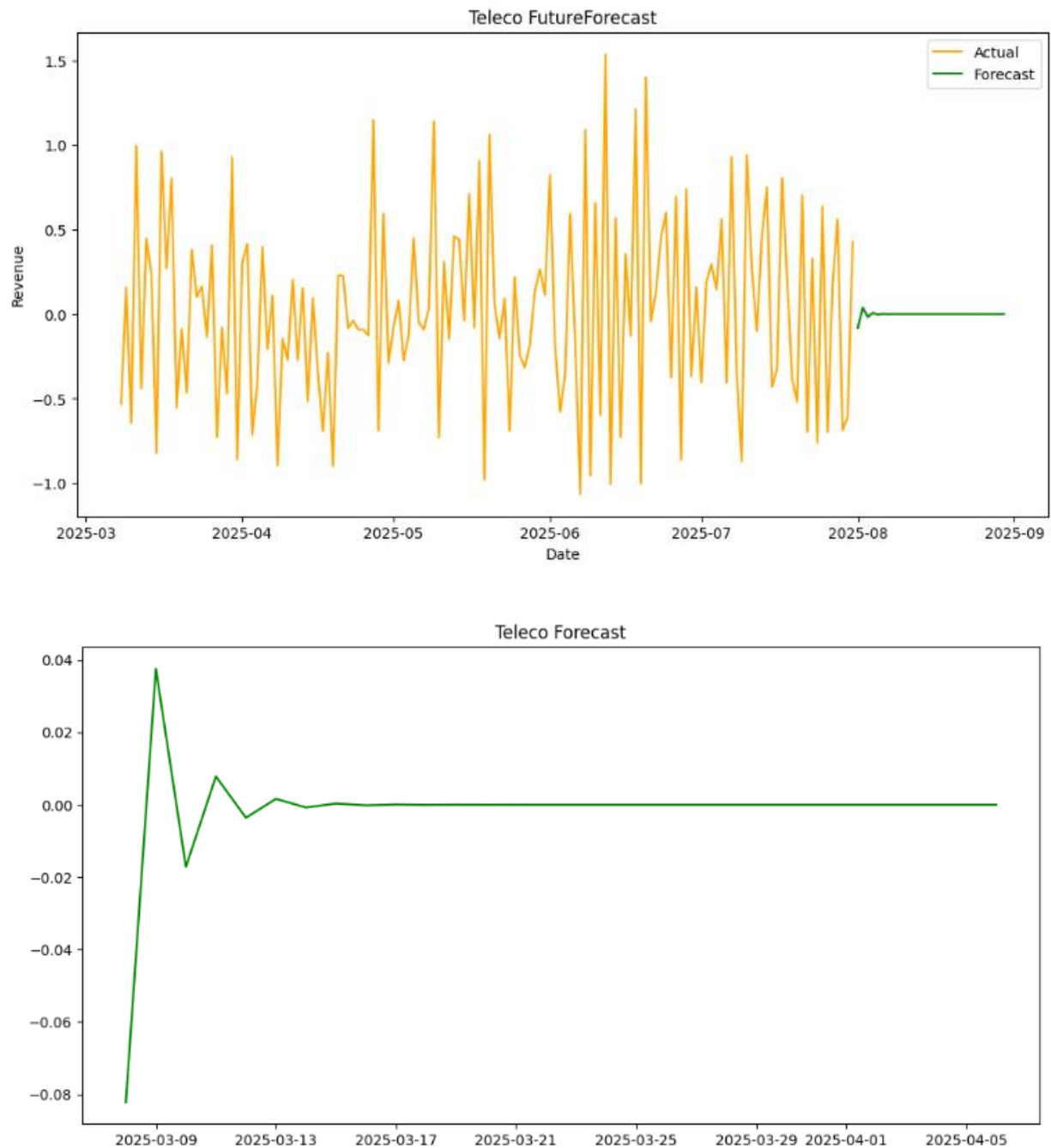
```
Mean Squared Error: 0.32, Mean Absolute Error: 0.47
RMSE: 0.5700310582332536
```

E2: ANNOTATED VISUALIZATION

Visualization of the forecast of the final model compared to the test set.

The test is the observed/actual value versus the forecast from the model. The forecast shows that the model is able to forecast the data in the short term, but not in the long term.





E3: RECOMMENDATION

Based on the confidence interval of 97.5%, the model can forecast revenue in the short term accurately with high accuracy, as confirmed by the MSE value of 0.32, the MAE of 0.47, and the RMSE of 0.57, showing that the model fits the data reasonably well in a span of 1 week but

not in the long run. The executive can use the model to understand the short-term trend and performance for the teleco business, but to be able to forecast for a longer duration, more analysis is needed.

F: REPORTING

The report is accurately created using a Jupyter Notebook, which is an industry-relevant interactive development environment for the Python programming language, and a PDF document of the executed notebook presentation is provided

G: SOURCES FOR THIRD-PARTY CODE

None

H: SOURCES

Brownlee, Jason (2020, August 14) A Gentle Introduction to Autocorrelation and Partial Autocorrelation

<https://machinelearningmastery.com/gentle-introduction-autocorrelation-partial-autocorrelation/>

Brownlee, Jason (2020, August 14). Introduction to Time Series Forecasting With Python

<https://machinelearningmastery.com/introduction-to-time-series-forecasting-with-python/>

Pandian, Shanthababu (2025, May 1). Time Series Analysis: Definition, Components, Methods, and Applications

<https://www.analyticsvidhya.com/blog/2021/10/a-comprehensive-guide-to-time-series-analysis/>

Intellectus Consulting (2025), Time series Analysis

<https://www.statisticssolutions.com/free-resources/directory-of-statistical-analyses/time-series-analysis/>

I: PROFESSIONAL COMMUNICATION